



ProgramDistill: From Interactive Web Apps to Verifiable Reference-Guided SWE Tasks

Jeonghye Kim¹, Minseon Kim², Young Jin Kim³, Matheus Pereira²
 Marc-Alexandre Côté², Xingdi Yuan², Alessandro Sordoni², Zhengyan Shi²
¹KAIST ²Microsoft Research Montréal ³Microsoft AI
 jeonghye.kim@kaist.ac.kr froggy@microsoft.com
<https://aka.ms/froggy/>

Coding agents are typically evaluated with desired behavior already specified through issues or instructions. In practical web development, however, agents may need to infer behavior from working software and implement it in an incomplete application. We introduce **ProgramDistill**, a benchmark that evaluates the capabilities of coding agents to implement features discovered through interaction with fully functional reference applications. We build ProgramDistill by factorizing reference applications into features of different granularities, each associated with replayable behaviors executable via the feature gold code patch. Our pipeline, **mine-craft-patch**, discovers 1,975 replay-verified behaviors across 26 applications and constructs 4,063 tasks without human intervention. Across nine frontier coding agents, GPT-6 Astra and Claude Opus 5 achieve 49.2% and 28.8% success on cumulative workflows in full-application reconstruction. In partial reconstruction, their success falls from 100% to 64.0% and from 96% to 32% as restoration depth increases from 1 to 8. ProgramDistill thus provides a scalable benchmark with a controlled progression of difficulty for evaluating and diagnosing coding agents, while providing a natural basis for future curriculum-based training.

1 Introduction

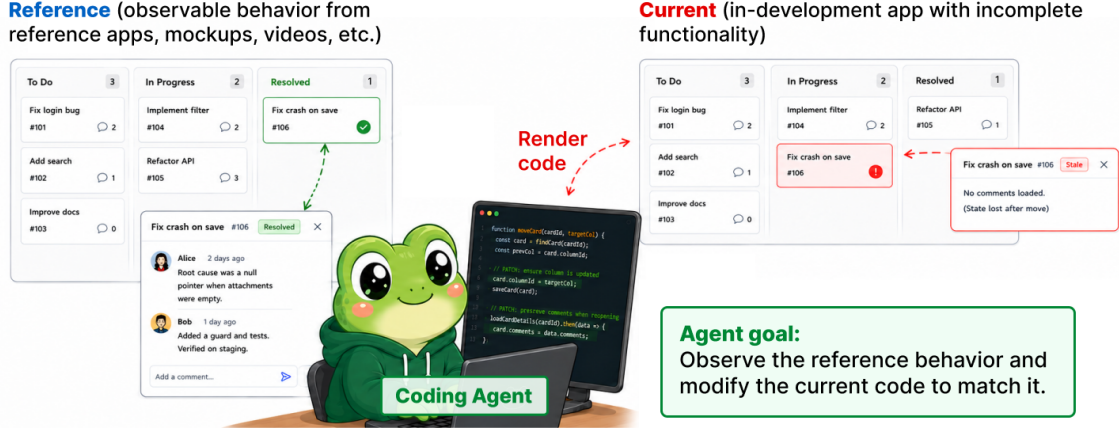


Figure 1: **ProgramDistill evaluates reference-guided software engineering for interactive web applications.** A coding agent is given a working **reference** application whose source implementation is hidden and an editable **current** application with missing functionality. The agent interacts with the reference to infer the intended behavior, restores that behavior by modifying the current implementation, and validates the result against the reference.

Large language models (LLMs) increasingly serve as coding agents capable of inspecting repositories and modifying source code in response to issues, instructions, or tests [15, 34, 27, 14, 33]. These settings, however, typically assume that the desired behavior has already been specified. In practical web development, developers may instead need to infer intended behavior directly from a working reference, such as an earlier product version, an interactive prototype, a comparable application, or a demonstration video. As illustrated in Figure 1, an agent must interact with the reference, modify the current implementation, and validate the resulting behavior against it. Even a simple interaction such as dragging a card may require inferring both its visible effect and the application state that persists afterward.

Recent work has begun to treat running software itself as part of the specification. ProgramBench [37], for example, asks agents to probe compiled programs in C/C++, Go, or Rust and reconstruct them from observable execution behavior. This establishes a behavior-to-code setting, but treats the program as a whole-program reconstruction target. Interactive

web applications expose additional structure. Their functionality unfolds through UI actions and application states, often with explicit behavioral prerequisites. A user may need to log in before creating an object, for instance, and create that object before modifying it later. Such stateful dependencies naturally organize behaviors into *prerequisite lineages*.

This raises a different question. **Can we uncover this latent behavioral structure and use it to turn a working interactive web application into a scalable source of verifiable software-engineering tasks?** Rather than treating an application as a single reconstruction problem, we identify features at varying levels of granularity and construct a SWE task for each, together with replayable interactions and a ground-truth code patch. These feature-level tasks can then be composed along prerequisite lineages into tasks of increasing depth, yielding a controllable axis of difficulty. We call this transformation *application-to-task factorization*.

The resulting tasks require a complementary form of behavioral distillation. Given a working *reference* and an incomplete *current* implementation, the agent must infer the required behavior from the reference and realize it through source-code changes in the current application. We call this *reference-to-current distillation*, where success is measured by verifying whether the replayable interaction executes correctly with the proposed code patch. Together, application-to-task factorization and reference-to-current distillation define **ProgramDistill**. Here, *distillation* broadly refers to extracting and transferring executable program behavior across these two transformations, while leaving downstream use open to evaluation or training.

To realize this framework at scale, ProgramDistill introduces a fully automated **mine–craft–patch** pipeline orchestrated by multiple LLM agents. It *mines* reproducible behaviors as replayable browser-interaction traces, *crafts* tasks by masking the source implementations responsible for those behaviors, and *patches* the resulting applications by asking coding agents to recover the missing functionality from a working reference. Each mined behavior serves as both a *task unit* and a *behavioral verifier*. The same trace succeeds on the intact application, fails after masking, and is replayed after repair. By preserving prerequisite relationships, ProgramDistill constructs atomic and cumulative tasks ranging from localized repair to full-application reconstruction. Across 26 applications, the pipeline mines 1,975 replay-verified behaviors and constructs 4,063 tasks without human-written issues, tests, or behavioral annotations.

We evaluate nine frontier coding agents. In full-application reconstruction, GPT-6 Astra [24] and Claude Opus 5 [3] recover 49.2% and 28.8% of evaluated workflows. In partial reconstruction, their success falls from 100% to 64.0% and from 96% to 32%, respectively, as restoration depth increases from 1 to 8. Trajectory analysis reveals a growing mismatch between reconstruction burden and agent effort, with observation effort per required behavior declining especially sharply as tasks deepen. Astra, meanwhile, achieves the strongest repair performance with the highest observation activity and the fewest edit/write steps. Together, these results identify effort allocation across observation, validation, and editing as an important dimension of reference-guided software engineering alongside implementation capability. ProgramDistill thus provides a structured and scalable benchmark with a controlled progression from partial repair to long-horizon reconstruction.

We make the following contributions.

- **A benchmark for reference-guided software engineering.** We introduce ProgramDistill, comprising 4,063 replay-verified SWE tasks derived from 1,975 behaviors across 26 interactive web applications, spanning atomic repair, cumulative repair, and full-application reconstruction.
- **Program distillation from executable behavior.** We formulate program distillation along two complementary axes: application-to-task factorization, which converts working applications into structured SWE tasks, and reference-to-current distillation, which requires agents to recover behavior from an executable reference. We operationalize these transformations with a fully automated mine–craft–patch pipeline.
- **Restoration depth for evaluation and curriculum construction.** Prerequisite lineages provide a controlled progression from atomic repair to full reconstruction, exposing systematic failures as restoration depth increases and providing a natural curriculum for future training.

2 Application-to-Task Factorization via the Mine–Craft–Patch Pipeline

Figure 2 illustrates **mine–craft–patch**, our new synthetic task generation pipeline that orchestrates multiple LLM agents to synthesize verifiable tasks from web applications without relying on human-written issues, tests, or behavioral annotations. The pipeline is model-agnostic, and we use GPT-5.6 Sol [23] for all stages in our experiments.

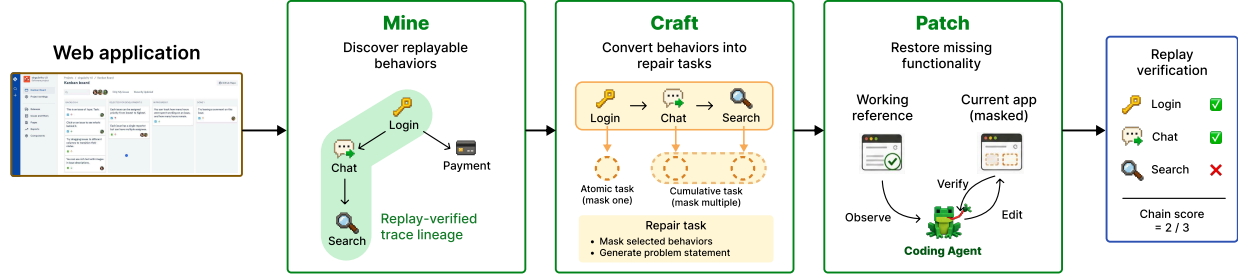


Figure 2: **Overview of the mine–craft–patch pipeline.** *Mining* discovers replayable behaviors and organizes them into a prerequisite trace tree by extending previously verified traces. *Crafting* masks individual behaviors or combines masks along a lineage to construct progressively deeper cumulative repair tasks. *Patching* asks a coding agent to restore the masked functionality from a working reference, then replays the corresponding lineage traces to measure how much of the behavior chain has been recovered.

2.1 Setup: Deterministic and Observable Application Instances

All stages of the pipeline operate on live, self-contained application instances through a common browser interface rather than on static source code alone. This setup provides a stable reference for observing behavior, an editable application for constructing and solving repair tasks, and a reproducible runtime for replay-based verification.

2.1.1 Applications and Runtime Instances

Application corpus. The mine-craft-patch pipeline operates on 26 web applications drawn from two complementary sources. The corpus includes self-contained applications adapted from the OSWorld web-application suite [32], as well as real-world open-source projects and SaaS clones drawn from public repositories. The former provide a controlled and diverse interaction substrate, while the latter contribute larger codebases, deeper workflows, nontrivial state, and heterogeneous architectures. Details of the corpus are provided in Appendix A.

Reference and editable instances. For each web application, we serve a fixed production build as the *reference instance* and a development-server version as the *editable instance*. The reference remains unchanged throughout task construction and repair, while masking and agent edits are applied to the source code and reflected in the editable instance through hot reload. Both instances expose the same application interface, allowing behavior observed in the reference to be reproduced and evaluated in the editable instance.

Deterministic execution. Reliable replay requires a reproducible starting state. The pipeline therefore resets application state before collection and replay and uses a shared deterministic clock across the database, backend, and frontend. This reduces variation from time-dependent behavior while preserving normal ordering and timeout semantics. Details of the deterministic execution setup are provided in Appendix A.1.

2.1.2 Browser Helper

A shared Playwright-based browser helper [18] provides a common interaction interface for mining, replay verification, and repair. It executes high-level actions and returns structured observations of visible text, accessibility information, and interactive elements. For reliable replay, it resolves elements using stable observable attributes rather than volatile DOM identifiers and waits for network activity and DOM mutations to settle before recording state. The same action and observation semantics are used throughout the pipeline. The helper also records screenshots for analysis and behavior relabeling, while the repair and reconstruction agents evaluated in this work receive only structured browser observations. Appendix B provides implementation details.

2.2 Mining: Discovering and Verifying Interactive Behaviors

As shown in Figure 2, the mining agent converts a running application into a bank of replay-verified behavioral traces. It combines four specialized LLM roles—a *Planner*, *Collector*, *Relabeler*, and *Reflector*—with replay-based verification. Appendix D provides details on these roles and their prompts.

Grounded behavior discovery. The mining process runs over multiple rounds. In round r , *Planner* agents inspect the live application and examine its source code using read-only tools. Using the current verified trace bank $\mathcal{B}^{(r)}$, they propose N new behavior goals, each accompanied by route hints grounded in UI observations and source evidence. A goal may specify a parent trace from $\mathcal{B}^{(r)}$ when the new behavior builds on the state reached at the end of that trace. For example, a trace that edits a newly created card can use the card-creation trace as its parent.

Each proposed goal is then assigned to a *Collector* agent. Consider a goal whose parent lies at the end of an existing trace lineage $L_{d-1} = (\tau_1, \dots, \tau_{d-1})$. The pipeline resets the application and browser and replays L_{d-1} to recreate the parent state. Starting from this state, the *Collector* pursues the new goal through live browser interaction and records the resulting interactions as an exploratory trace $\tilde{\tau}_d$. For a goal without a parent, the *Collector* starts directly from the reset state.

During collection, the environment records browser actions and provides candidate success signals derived from the observed application state. After reaching a coherent outcome, the *Collector* selects supporting signals without authoring or modifying their content. A trace stores the action sequence with replay-stable selectors, the selected expected signals, and an optional parent reference. An example trace representation is provided in Appendix D.5.

After the *Collector* reaches the target behavior, the pipeline re-collects it from a clean state. Any prerequisite traces are replayed first, and the exploratory trace $\tilde{\tau}_d$ is included in the prompt as additional route guidance for the *Collector*. The *Collector* reproduces the behavior while omitting unnecessary exploratory actions, yielding a cleaned trace τ_d . This re-collection procedure is related to the use of privileged information in self-distillation, where richer conditioning can produce more concise trajectories [16]. Appendix D.2 provides further details.

The cleaned trace τ_d is then passed to the replay verifier described below. If it passes, the *Relabeler* receives the executed actions and observed outcome evidence, together with an animated GIF showing every retained step, but not the original goal. It assigns a name and description to the behavior actually achieved. The accepted trace extends the parent lineage to $L_d = (\tau_1, \dots, \tau_d)$. A root trace yields $L_1 = (\tau_1)$.

After all goals in the round have been processed, the accepted traces are added to the trace bank to form $\mathcal{B}^{(r+1)}$. The *Reflector* then summarizes the current coverage, observed failures, and promising directions for the next round. Table 1 summarizes the four roles, and Figure 3 shows their control flow. The resulting lineages support repair-task composition at different restoration depths in Section 2.3.

Table 1: Specialized sub-agents within the mining agent.

Sub-agent	Role
Planner	Proposes behavior goals and route hints grounded in application evidence.
Collector	Pursues goals, records interactions, and selects observed success signals.
Relabeler	Names the behavior actually achieved from replay-verified evidence.
Reflector	Summarizes outcomes to guide later rounds.

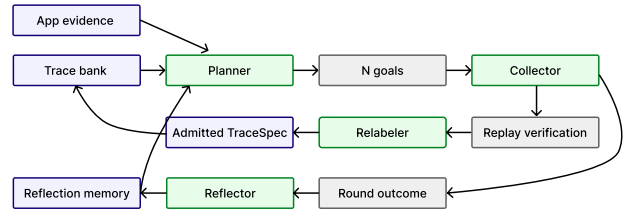


Figure 3: Mining-agent control flow. Each round proposes goals, collects and refines candidate traces, admits only replay-verified behaviors, and uses reflection to guide the next round.

Replay-based verification. For the intact application A and a trace lineage $L_d = (\tau_1, \dots, \tau_d)$, we define the replay verifier $V(A, L_d) \in \{0, 1\}$.

Starting from a reset application and browser state, the verifier replays $\tau_1, \dots, \tau_d$ in order without LLM intervention. It returns 1 only if every trace completes its recorded actions and satisfies its expected-signal assertions.

The recorded actions, selectors, and expected signals define the trace-level behavioral verification contract. UI properties such as roles, labels, and visible text are part of this contract when the recorded selectors rely on them.

A candidate trace τ_d is admitted only if $V(A, L_d) = 1$. The same verifier is used for mask validation and repair evaluation in Sections 2.3 and 2.4.

2.3 Crafting: From Verified Behaviors to Repair Tasks

Crafting turns replay-verified behaviors into repair tasks along two dimensions. *Mask scope* determines whether a task removes only the implementation logic or both the logic and UI. *Task composition* determines whether the task targets one behavior or multiple behaviors along a prerequisite lineage. Each behavior is first used to construct and validate an atomic task. Validated atomic masks are then composed along prerequisite lineages to form cumulative tasks.

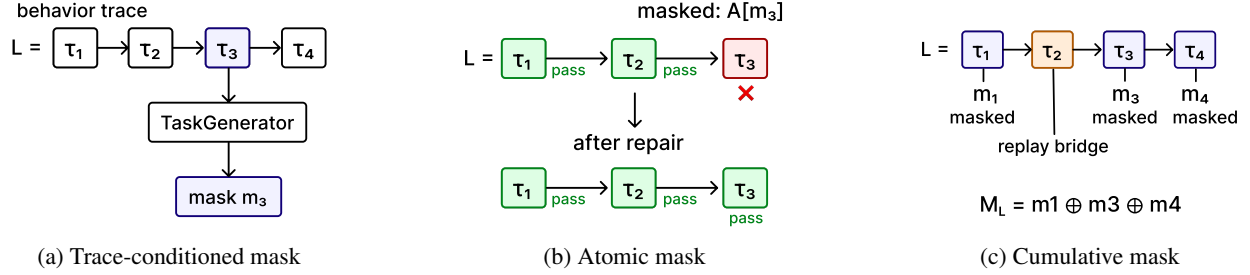


Figure 4: **Crafting verified behaviors into repair tasks.** (a) A behavior-specific mask is generated from a verified trace. (b) Atomic validation requires prerequisite traces to pass and the target trace to fail after masking. Gold-patch replay verifies recovery of the complete lineage. (c) Validated masks are composed along a prerequisite lineage, with unmasked traces serving as replay bridges and masked behaviors forming joint restoration targets.

Trace-conditioned masking. Consider a verified lineage $L = (\tau_1, \dots, \tau_d)$ with target trace τ_d . The pipeline identifies the source implementation responsible for the observed behavior of τ_d and proposes a mask m_d . We write $A[m_d]$ for the application obtained by applying m_d to the intact application A .

A *logic-only* mask removes the feature’s implementation logic while preserving its UI. A *logic-and-UI* mask also removes the UI owned by the feature. For each mask, the pipeline generates a concise behavioral problem statement describing the feature’s user-facing purpose and workflow, but not the original implementation, repair procedure, or exact verification signals. The coding agent must infer the missing behavior from the working reference. Task-generation prompts are provided in Appendix E.

Atomic task validation. An *atomic repair task* applies only the target mask m_d , leaving the implementations required by the preceding traces unmasked. Since L was replay-verified on the intact application, $V(A, L) = 1$ by construction. A proposed mask is accepted only if the masked application builds and launches successfully, the prerequisite lineage L_{d-1} remains replayable, and the target behavior fails. For $d > 1$, these conditions are

$$\text{Build}(A[m_d]) = 1, \quad V(A[m_d], L_{d-1}) = 1, \quad V(A[m_d], L_d) = 0.$$

For a root trace with $d = 1$, only the build and target-failure conditions apply.

These conditions induce an SWE-bench-style fail-to-pass (F2P) restoration objective for the target and pass-to-pass (P2P) preservation checks for its unmasked prerequisites [15].

A separate LLM-based mask-depth critic rejects superficial changes, such as toggling feature flags or removing call sites while leaving the substantive implementation intact. Failed proposals may be revised through bounded feedback rounds. Only masks that pass both counterfactual validation and the critic are retained. If no proposed mask is accepted, the trace remains available as a prerequisite but does not become a repair target.

Cumulative task composition. For a verified lineage $L = (\tau_1, \dots, \tau_d)$ ending at a repair target, let $\mathcal{M}_L$ denote the ordered collection of validated masks along the lineage

$$\mathcal{M}_L = (m_{j_1}, \dots, m_{j_{r_L}}), \quad 1 \leq j_1 < \dots < j_{r_L} = d.$$

The *restoration depth* $r_L = |\mathcal{M}_L|$ is the number of behaviors to be restored along the lineage. The cumulative mask is

$$M_L = m_{j_1} \oplus \dots \oplus m_{j_{r_L}},$$

where $\oplus$ combines the accepted source modifications into a single runnable masked application.

Every masked behavior along the lineage is a restoration target, while unmasked traces serve as *replay bridges* that establish prerequisite state without becoming repair targets themselves. Thus, lineage depth d may exceed restoration depth r_L . Atomic tasks have $r_L = 1$, whereas cumulative tasks have $r_L > 1$.

The pipeline first attempts deterministic composition. Because all component masks are defined against the same intact baseline A , non-overlapping edits are combined directly and overlapping deletions are merged by union. A `git` three-way merge provides an independent consistency check. If the two results disagree, deterministic composition fails, and an LLM-based merge agent resolves the remaining overlaps while preserving the non-conflicting masks. Its prompt is provided in Appendix E.3. Of the 1,201 cumulative tasks in our experiments, 629 were composed deterministically and 572 required the merge agent. The share requiring the merge agent increases with restoration depth, from 25.6% at $r_L = 2$ to 89.7% at $r_L = 8$ and 100% at $r_L > 8$. All 572 agent-assisted merges succeeded without omitting any intended mask component.

Reference restoration patches. For both atomic and cumulative tasks, reversing the final masking diff yields a *reference restoration patch*, or *gold patch* [15], that recovers the original implementation. We validate each gold patch by applying it to the masked application and requiring the complete task lineage to pass replay verification. Atomic counterfactual checks provide target-specific negative controls, while gold-patch replay provides an end-to-end positive control that the restoration and preservation requirements can be satisfied jointly.

2.4 Patching: Restoring Behavior from a Working Reference

The validated repair tasks produced during crafting are then passed to coding agents for reference-guided restoration. Each task pairs a masked current application with a working reference application that exposes the target behavior.

Reference-guided repair. The coding agent receives the masked repository and a generated behavioral problem statement. It edits a writable, hot-reloading `current` application while interacting through the browser with a working `reference` application whose source is hidden. The masking diff, gold patch, and grading traces are withheld from the agent, and public-network egress is disabled to block public-source retrieval. Section 3.2.2 examines observed probes for hidden implementations and evaluation artifacts. The runtime layout and permitted communication paths are described in Appendix C.1. An example agent instruction is provided in Appendix F.

Trace-level verification and lineage scoring. For scoring, the submitted patch is applied to the task’s masked application, which is then rebuilt and launched. Evaluation uses the verifier defined in Section 2.2, with the recorded actions, selectors, and expected signals unchanged. Both repair targets and replay bridges must satisfy the recorded action and outcome checks, enforcing the restoration and preservation requirements established during Crafting (Section 2.3). The submitted patch need not match the gold implementation at the source level. Function names, variable names, and code structure may differ, as long as the repaired application reproduces the reference behavior.

To score how much of the restoration lineage is recovered, replay proceeds in lineage order until the first failure. For each repair target τ_{j_k} , where $k = 1, \dots, r_L$, let $p_k \in \{0, 1\}$ indicate whether replay succeeds through τ_{j_k} . Targets not reached after an earlier failure receive $p_k = 0$. We define

$$\text{BinaryScore}(L) = \prod_{k=1}^{r_L} p_k, \quad \text{ChainScore}(L) = \frac{1}{r_L} \sum_{k=1}^{r_L} p_k. \quad (1)$$

The binary score is 1 only when all repair targets in the lineage are recovered. The chain score gives partial credit for the recovered prefix before the first failure. For an atomic task, $r_L = 1$, so the two scores coincide.

Full-application reconstruction. So far, we have considered partial repair, where the agent restores missing behaviors in an existing application. We also extend the same reference-guided setting to full-application reconstruction, where the agent starts from a minimal executable scaffold.

To evaluate whether the application has been reconstructed correctly, we reuse the replay-verified behavior traces collected during mining. Specifically, we sample traces from the verified trace bank $\mathcal{B}^{(r)}$ to form a behavioral test suite and replay them on the reconstructed application using the same verifier. Individual-behavior recovery measures the fraction of sampled traces that pass together with their prerequisites, while cumulative workflows use the lineage scores defined above. Because reconstruction starts from a scaffold rather than an existing implementation, prerequisite replay checks workflow correctness rather than preservation of previously working behavior.

3 Experiments

3.1 Benchmark Generation Statistics

We run the full **mine-craft-patch** pipeline over 26 web applications using GPT-5.6 Sol as the construction model. Figure 5 summarizes the resulting mining yield, task composition, and mask scopes.

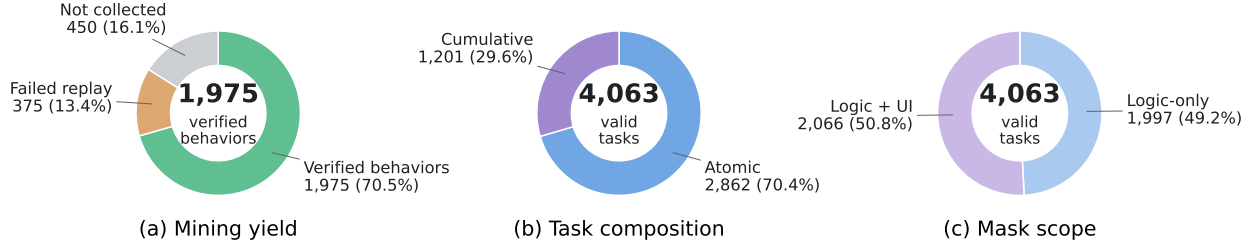


Figure 5: **ProgramDistill benchmark generation.** (a) Mining yields 1,975 replay-verified behaviors, with remaining goals either uncollected or failing replay verification. (b) The resulting 4,063 repair tasks comprise atomic and cumulative tasks. (c) Tasks are distributed across logic-only and logic-and-UI mask scopes.

Mining. Across the 26 applications, the Planner proposes 2,800 candidate goals, of which 2,350 are collected as interaction traces. After clean-state replay, 2,165 traces reproduce successfully, and 1,975 survive the final replay check to become verified behavior traces. The mined prerequisite trees reach a maximum *lineage depth* d of 17 (mean 3.40), a maximum width of 60, and a mean branching factor of 1.78. Lineage depth is the number of traces from the root to a target, including replay bridges used to establish prerequisite state.

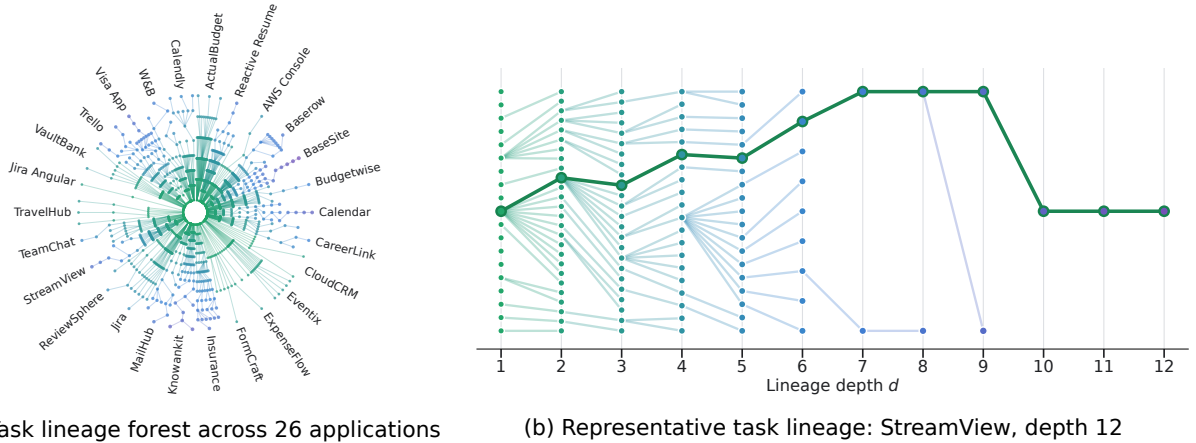


Figure 6: **Prerequisite lineages across applications.** (a) Mined lineage forests across all 26 applications. (b) A representative StreamView lineage reaching depth 12, with one root-to-leaf prerequisite chain highlighted.

Lineage. As described in Section 2.3, cumulative tasks are constructed by composing masks along mined prerequisite lineages. Figure 6 visualizes this structure at two scales. Panel (a) shows variation in lineage depth and branching across applications, reflecting the range of dependent workflows discovered during mining. Panel (b) shows a representative StreamView lineage reaching depth 12, with columns indexing lineage depth d and one root-to-leaf chain highlighted.

Crafting. The pipeline constructs 4,063 repair tasks from the verified traces: 2,862 atomic tasks and 1,201 cumulative tasks. Across both task types, 1,997 use logic-only masks and 2,066 use logic-and-UI masks. Not every trace in a mined lineage becomes an independent repair target. Traces that do not satisfy the task-admission criteria in Section 2.3 are retained as replay bridges to establish prerequisite state. Accordingly, as defined in Section 2.3, the *restoration depth* r_L counts the lineage behaviors that become repair targets and whose code must be restored.

ProgramDistill-300. Evaluating all 4,063 tasks for every model would be prohibitively expensive, so we define a fixed 300-task evaluation suite. We stratify tasks by restoration depth r_L using quotas of 50, 45, 45, 40, 35, 30, 30, and 25 for depths 1 through 8. Within each depth, tasks are selected round-robin across applications to preserve coverage of all 26 apps and prevent applications with larger task pools from dominating. The resulting suite covers 269 distinct lineages and contains 50 atomic and 250 cumulative tasks, with 140 logic-only and 160 logic-and-UI masks.

3.2 Partial-Application Reconstruction

We evaluate nine frontier models—GPT-6 Astra [24], GPT-5.6 Sol [23], Claude Opus 5 [3], Claude Sonnet 5 [4], Gemini 3.7 Flash [9], Gemini 3.6 Flash [8], Gemini 3.1 Pro Preview [7], Grok 4.6 [30] and GPT-5.3 Codex [22]—on ProgramDistill-300, with reasoning effort set to high. Each model receives a masked application and a problem statement and must recover the missing behavior by inspecting a reference, editing the current implementation, and validating its changes against the live application. Depth-1 tasks are atomic repairs, while depths 2 through 8 combine multiple repair targets from a prerequisite lineage.

Agent harness. All models use the same modified R2E-Gym [14] agent harness. The harness exposes four coding tools: `execute_bash`, `file_editor`, `search`, and `finish`. Through `execute_bash`, the agent can also invoke the browser CLI to observe and interact with both the masked current application and the working reference. Appendix C describes the harness design and execution settings.

3.2.1 Model Performance Comparison

We evaluate nine models on ProgramDistill-300, comparing repair performance with model cost. Figure 7(a) plots mean binary score against list output price, while Appendix H.2 reports the corresponding chain-score results. GPT-6 Astra achieves the highest mean binary score at 84.3%, followed by Claude Opus 5 at 68.7% and GPT-5.6 Sol at 60.7%. These are also the three most expensive models by list output price, with Astra gaining 15.7 percentage points over Opus 5 at twice the output-token price. Grok 4.6, Claude Sonnet 5, GPT-5.3 Codex, and Gemini 3.7 Flash form a middle tier, with scores between 45.3% and 48.3%. Gemini 3.6 Flash and Gemini 3.1 Pro are the least expensive models, with scores between 25.3% and 28.3%.

Higher-priced models tend to occupy the top of the performance range, but price alone does not determine repair performance. Gemini 3.7 Flash nearly matches GPT-5.3 Codex (45.3% vs. 45.7%) at roughly one quarter of its output-token price, while Grok 4.6 slightly exceeds Claude Sonnet 5 (48.3% vs. 47.3%) at a lower list price.

Mask scope provides a complementary view of repair difficulty. Astra’s chain score is 96.2% on *logic-only* tasks and 84.9% on *logic-and-UI* tasks. The 11.3-point gap shows that interface reconstruction remains harder even when partial restoration receives credit. Appendix H.1 reports the full scope comparison.

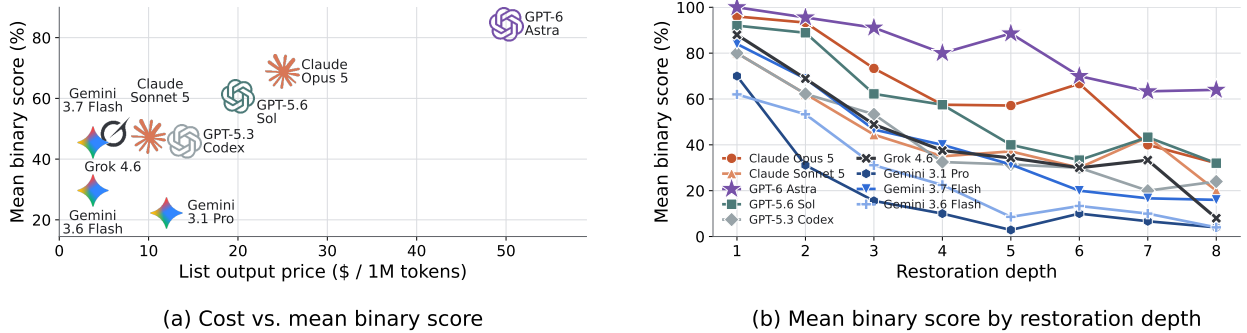


Figure 7: **Partial-application reconstruction performance.** (a) Mean binary score versus list output price (\$/1M tokens) from the LiteLLM `model_cost` table (Table 8 in Appendix G.1). (b) Mean binary score by restoration depth r_L . Performance generally declines as depth increases. Figure 18 in Appendix H.2 shows the same panels under the chain score.

Degradation with restoration depth. Astra is the only model to solve all depth-1 tasks, achieving 100% binary success, compared with 96% for Opus 5 and 92% for Sol (Figure 7(b)). Because cumulative tasks are composed of individually validated atomic repairs and are themselves verified with gold patches, this depth-1 result establishes a

strong empirical baseline for task solvability. Performance nevertheless drops as multiple dependent repairs must be recovered together. Astra scores 64.0% at depth 8, a 36-point decline, while Opus 5 and Sol both score 32.0%. Every other model retains less than half of its depth-1 performance. These results suggest that restoration depth exposes a compositional challenge, as agents can often solve individual repairs but struggle to combine multiple repairs while preserving their dependencies. Appendix H.3 examines this pattern at similar mean prompt sizes per step.

Example of a successful deep repair trajectory. Although performance declines with restoration depth, some deep tasks are successfully repaired through sustained iterative refinement. Figure 8 shows one such Claude Opus 5 trajectory on a depth-8 cumulative repair task in Trello (Vdevired). The agent first inspects the reference and repository to recover the core functionality, then repeatedly revisits the reference and current application to identify residual mismatches, apply targeted fixes, and validate them. The trajectory illustrates how a successful deep repair can require repeated observe–edit–validate cycles across multiple dependent behaviors.

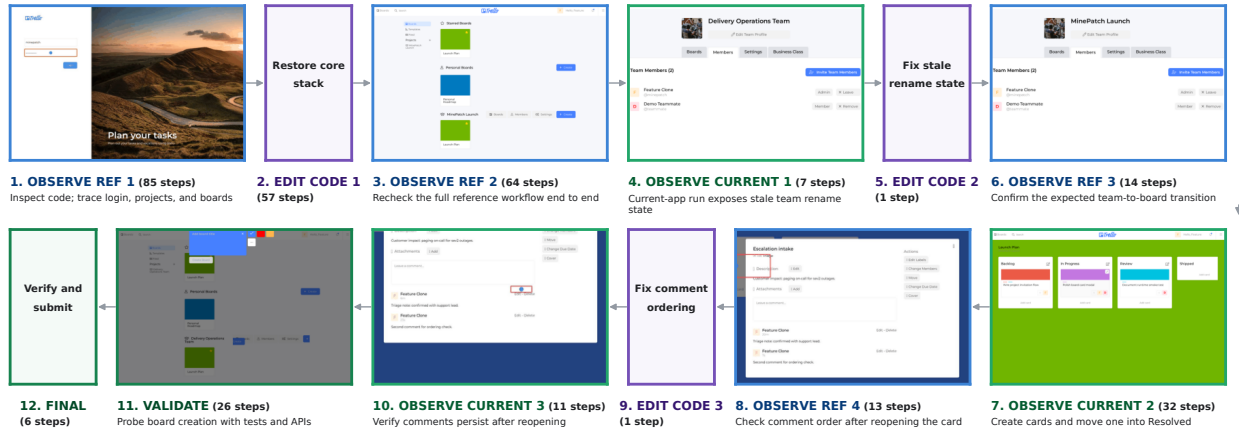


Figure 8: **Reference-guided repair on a depth-8 Trello (Vdevired) task.** The depth-8 task spans eight dependent workflow stages—login, project setup, team and board setup, list and card creation, permissions, card details, cross-list movement, and comment editing—all of which must remain functional together. Across 317 steps, Claude Opus 5 repeatedly alternates between **OBSERVE REF** to infer intended behavior, **EDIT CODE** to modify the implementation, and **OBSERVE CURRENT** to validate the repair before final submission.

3.2.2 Analysis

To characterize repair workflows, we compare the recorded trajectories of all nine models on ProgramDistill-300.

Model-level behavior. Models exhibit markedly different interaction patterns during repair. Figure 9 compares reference and current-app observation, repository read/search, and edit/write activity alongside repair performance.

Astra stands out for an observation-intensive, edit-light workflow. It records the most reference and current-app observation steps, with the largest lead in current-app observation: 96.3 steps per trajectory, approximately $2.1\times$ the next-highest mean (Sol, 45.8). It achieves the strongest repair performance while making the fewest edit/write steps (9.9 per trajectory). Opus 5 and Sonnet 5 are also relatively observation-intensive, but pair this with more editing (23.5 and 15.7 steps, respectively). This profile is consistent with intensive behavioral checking, particularly of its own implementation, coupled with selective code changes. More broadly, these results suggest that repair performance may depend not only on implementation capability but also on **how effectively agents allocate effort across observation, validation, and editing**. Table 13 in Appendix H.4 reports interaction statistics across models and restoration depths.

Scaling with restoration depth. To better understand why repair performance declines with restoration depth, we examine how task burden and agent effort scale as more behaviors must be restored. Figure 10 shows a growing mismatch between the two. From depth 1 to depth 8, the mean number of code lines to restore increases from 139.8 to 1,305.2, accompanied by an increase in mean reference-trace length from 10.1 to 107.6 browser actions.

In contrast, agent effort per repair target decreases, with the sharpest contraction in behavioral observation. Reference observation steps per target fall from 34.60 to 8.46, and current-app observations from 27.69 to 6.81, reductions of

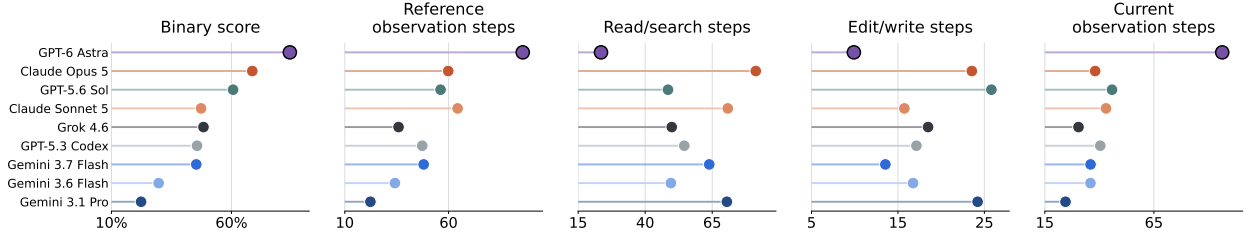


Figure 9: **Model-level performance and interaction behavior.** Mean binary score alongside mean reference observation, read/search, edit/write, and current-app observation steps per trajectory.

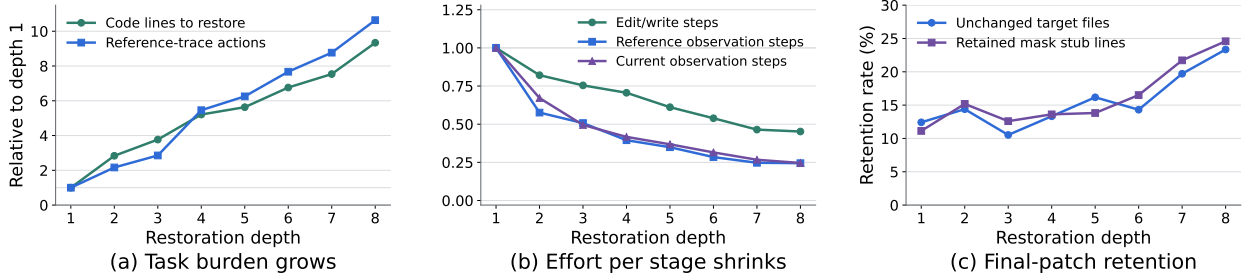


Figure 10: **Scaling of repair burden and agent effort with restoration depth r_L .** (a) Repair burden increases with restoration depth. (b) Agent effort per repair target, measured by edit/write and reference/current-app observation steps. (c) Final-patch retention of unchanged target files and mask-added stub lines. Panels (a) and (b) are normalized to their depth-1 values, while panel (c) reports raw percentages.

roughly 75% in both cases. Edit/write steps also decline, but more moderately, from 7.53 to 3.40. **Thus, as restoration depth grows, agents devote disproportionately less effort to understanding and validating each required behavior.**

Final patches also leave more of the masked implementation unrestored as restoration depth increases. Between depths 1 and 8, the mean fraction of unchanged target files changes from 12.4% to 23.3%, while the fraction of mask-added stub lines retained in the final patch changes from 11.1% to 24.6%. Although intermediate depths fluctuate, the overall pattern suggests that deeper tasks leave more required implementation unrecovered. Together with the decline in binary score and the reduced observation and editing effort per target, this indicates that agents struggle to keep pace as reconstruction burden grows.

Shortcut attempts. Beyond the intended repair workflow, agents may attempt to recover implementation information from sources outside the supplied code and permitted reference interface. We therefore identify access attempts involving Git history, public app-source retrieval, protected harness or evaluator files, and direct reference access.

Figure 11(a) shows the frequency of public app-source retrieval, Git history, and other identified access channels. At least one such attempt appears in 7.2% of trajectories, with substantial variation across models. Grok 4.6 has the highest rate at 19.0%, followed by Gemini 3.1 Pro at 17.7%, compared with 0.3% for Astra.

These channels are blocked by design through several safeguards. Workspaces contain no pre-mask Git history, stale artifacts are regenerated from the masked source, public egress is disabled, harness and grading files are hidden or protected, and the reference is accessible only through the browser helper. Details of the runtime architecture and isolation safeguards are provided in Appendix C.1.

3.3 Full-Application Reconstruction

We next evaluate the full-application reconstruction setting introduced in Section 2.4. Starting from a minimal executable scaffold, the agent must rebuild the application by using a product-level capability description and interacting with a live reference. We evaluate this setting on twelve stateful web applications using GPT-6 Astra, Claude Opus 5, and GPT-5.6 Sol at max reasoning effort. The harness configuration is described in Appendix C.

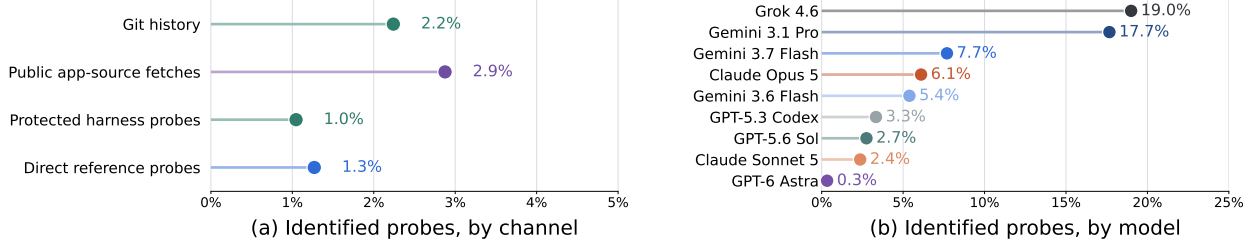


Figure 11: **Identified shortcut attempts.** (a) Share of trajectories with an identified attempt in each channel. (b) Within-model share with at least one identified attempt.

Evaluation suite and scoring. Following the protocol in Section 2.4, we evaluate full-application reconstruction using two types of behavioral tests. Individual-behavior tests check whether individual replay-verified behaviors are reproduced, while cumulative workflow tests assess whether multiple dependent behaviors are recovered together. The evaluation suite contains 590 individual behaviors and 413 cumulative workflows across twelve applications.

3.3.1 Model Performance Comparison

Table 2: **Full-application reconstruction performance.** Atomic recovery measures reproduction of individual replay-verified behaviors, while cumulative recovery requires all target behaviors in a workflow to pass. Bold indicates the best result among three models. Appendix H.2 reports the corresponding cumulative chain scores.

Model	Reactive Resume	AWS Console	Baserow	BudgetWise	CareerLink	MailHub	Jira (Oldboyxx)	Trello (Knowankit)	StreamView	TeamChat	Trello (Vdevired)	Weights & Biases	Mean
<i>Atomic behavior recovery</i>													
GPT-6 Astra (max)	50.0	56.0	14.0	60.0	62.0	88.0	64.0	53.3	82.0	68.0	28.9	78.0	58.98
Opus 5 (max)	20.0	16.0	10.0	48.0	42.0	92.0	82.0	55.6	32.0	58.0	6.7	40.0	42.03
GPT-5.6 Sol (max)	8.0	8.0	8.0	30.0	18.0	78.0	72.0	55.6	32.0	50.0	26.7	16.0	33.39
<i>Cumulative binary score</i>													
GPT-6 Astra (max)	43.8	45.8	5.9	42.6	55.6	81.8	75.0	38.1	75.9	45.5	23.5	72.7	49.15
Opus 5 (max)	17.2	25.0	2.9	21.3	27.8	86.4	86.1	38.1	13.8	33.3	0.0	24.2	28.81
GPT-5.6 Sol (max)	4.7	0.0	2.9	12.8	8.3	59.1	75.0	47.6	24.1	27.3	20.6	3.0	21.07

Table 2 shows that full-application reconstruction remains challenging even for the strongest models. GPT-6 Astra performs best on both individual behaviors and cumulative workflows, reaching 58.98% atomic recovery and 49.15% cumulative recovery. Claude Opus 5 achieves 42.03% atomic and 28.81% cumulative recovery, while GPT-5.6 Sol achieves 33.39% and 21.07%, respectively.

Notably, this ordering matches the masked-repair results, with Astra outperforming Opus 5 and Opus 5 outperforming Sol in both settings. The consistency across substantially different task regimes suggests that ProgramDistill captures stable differences in reference-guided software engineering capability rather than effects specific to one task formulation.

Across models, cumulative recovery is consistently lower than atomic recovery, showing that reproducing individual behaviors does not necessarily translate into recovering complete multi-step workflows. Performance also varies sharply across applications. For Astra, cumulative recovery ranges from 81.8% on MailHub to 5.9% on Baserow. One possible explanation is that some applications expose more regular and repetitive interaction structures, while others require coordinating more heterogeneous state and workflows. MailHub, for example, is largely organized around recurring message and thread operations. In contrast, lower-scoring applications involve more varied representations, such as user-defined table schemas in Baserow, resume layouts and JSON import/export in Reactive Resume, service-specific resource configurations in AWS Console, and plan and SIM selections across cart and checkout in BudgetWise. Maintaining consistency across these heterogeneous representations may make full-application reconstruction substantially harder.

Observation strategies in reconstruction. Unlike partial repair, where trajectories average roughly 200 agent steps, full-application reconstruction produces much longer runs, averaging around 700 steps and reaching up to 1,921 (Table 3). In this setting, Opus 5 often uses shell for loops to issue multiple browser CLI calls within a single agent step, effectively collecting several browser states at once. This yields 7.9 browser state returns per agent step on average, compared with 1.6 for Astra and 1.2 for Sol.

Table 3: **Agent activity during full-application reconstruction.** Min, mean, and max per-run counts of agent steps, observation steps, and browser state returns. Observation steps use the same request-based definition as in the repair analysis. Browser state returns count recorded helper responses, including those returned by interactions and failed actions, and multiple responses may occur within one step.

Model	Steps			Observation steps			Browser state returns		
	Min	Mean	Max	Min	Mean	Max	Min	Mean	Max
GPT-6 Astra (max)	292	719.4	1921	221	643.3	1796	406	1154.1	3843
Opus 5 (max)	371	627.5	940	59	258.2	557	1427	4932.1	12 821
GPT-5.6 Sol (max)	348	719.0	1165	204	419.3	705	277	854.7	1441

Distinguishing observation steps from browser state returns exposes this difference in interaction style. Despite the much larger volume of browser feedback, Opus 5 still recovers fewer behaviors than Astra, suggesting that observation volume alone does not explain reconstruction performance.

3.3.2 Analysis

Full-application reconstruction remains difficult even for the strongest agents. We therefore ask why agents fail to recover behaviors that are available to inspect in a working reference. We analyze 977 failed atomic behaviors across the 36 reconstruction runs in Table 2, using the submitted source, and recorded reference and current observations with GPT-6 Astra. Each failure is assigned to one of four categories: not observed, observed but not implemented, implemented with the wrong observable form, or implemented with the wrong state, route, or result. We also examine why the agent’s own validation did not expose and correct the remaining difference.

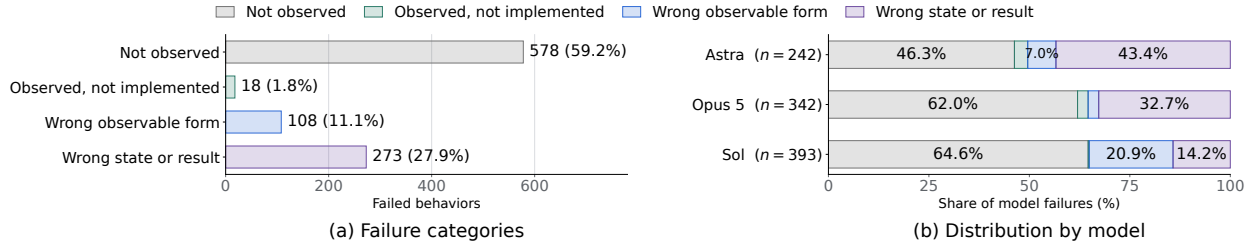


Figure 12: **Full-application reconstruction failures.** Across 36 runs on 12 applications, (a) counts failures in the four categories and (b) shows their distribution within each model’s failures.

Behavior-level reconstruction outcomes. As shown in Figure 12, the largest failure category is behavior that was not observed in the reference, accounting for 59.2% of failures. The remaining failures occur after the relevant behavior has been observed. Of these, 1.8% are observed but not implemented, 11.1% are implemented with the wrong observable form, and 27.9% produce the wrong state, route, or result. These outcomes reveal difficulties both in exploring the reference and in faithfully reproducing behaviors that have already been observed.

What validation missed. Agents checked that their reconstructed applications worked, but these checks did not always establish faithful reproduction of the reference behavior. In the failure cases in Figure 13, agents often validated related functionality or a limited set of workflows without rechecking the failing workflow after the final relevant source edit. For example, validating native resume creation and export did not reveal a failure in external JSON Resume import, while syntax-only checks did not reveal that report edits in W&B were not persisted.

Checks such as syntax validation, successful backend requests, or selected text matches can establish that an application runs without showing that its behavior matches the reference. As a result, omitted behaviors, incorrect observable forms, and wrong state transitions can remain undetected even when the relevant reference behavior has already been observed. A related pattern appears in partial reconstruction, where current-app observation effort per target decreases as restoration depth increases (Figure 10).

Example failure cases. Figure 13 illustrates four reconstruction errors that remain after agent validation. For each case, we replay the same action sequence on the reference and the final submission to expose the behavioral mismatch.

Across the four examples, the agent’s validation does not recheck the exact workflow that reveals the remaining error after its final relevant edit.

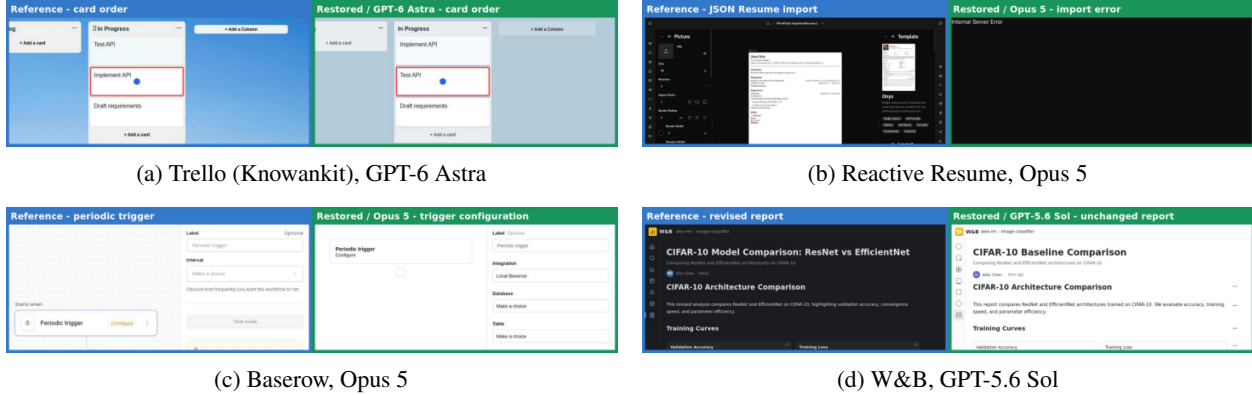


Figure 13: **Reconstruction errors missed during validation.** In each panel, the reference application appears on the left and the final submission on the right. **(a)** Dragging a card leaves the cards in the wrong order. **(b)** Importing a JSON Resume produces a rendering failure. **(c)** A periodic trigger shows database settings instead of an interval setting. **(d)** Changes to a report’s title and summary are not saved.

Figure 13 presents four representative reconstruction failures that remain after agent validation.

In Trello (Knowankit), Astra reconstructs card dragging, but a center drop leaves the cards in the opposite order from the reference. After the final relevant edit, the agent validates other board interactions rather than this drag workflow, so the ordering error remains undetected.

In Reactive Resume, Opus 5 implements JSON Resume import without converting the uploaded schema into the representation expected by the application, causing the imported resume to fail during rendering. Its final validation covers native resume creation, editing, and export, but does not exercise import of the external JSON Resume schema.

In Baserow, a trigger determines how an automated workflow starts. A periodic trigger should expose an `Interval` setting that specifies how often the workflow runs. Opus 5 changes the trigger title to `Periodic trigger`, but leaves the underlying database-trigger form in place. The reconstructed application therefore shows `Integration`, `Database`, and `Table` instead of the expected `Interval` setting. The agent validates a database-trigger workflow but does not recheck the periodic-trigger configuration.

Finally, in W&B, Sol renders a report editor but does not persist changes to the title and summary. After the same editing sequence, the reference retains the revised report while the reconstructed application still shows the original text. The agent’s final checks cover syntax without reopening the edited report, so the failed state update remains undetected.

4 Related Works

Repository-level software-engineering benchmarks. A large body of work evaluates coding agents on repository-level software engineering. SWE-bench [15] asks agents to resolve real GitHub issues, while R2E-Gym [14], SWE-smith [36], and related pipelines scale executable task construction from commits, tests, and source repositories, and open agent platforms such as OpenHands [28] standardize how such agents are built and run. DeepSWE [13] pushes the same setting toward long-horizon engineering work, where a single task spans many dependent changes. SWE-bench Multimodal [35] augments issue descriptions with visual evidence, and SWE-Together [29] extends this setting to multi-turn sessions in which users progressively clarify or correct requirements. These benchmarks increasingly capture realistic repository editing and execution, but the desired behavior is still supplied through an issue, test-derived task, visual artifact, or user interaction. ProgramDistill instead requires the agent to recover the specification by interacting with a working application, and uses the application’s own executable behaviors as both repair targets and replayable verifiers.

Visual and interactive web development. Web2Code [38] and Design2Code [26] study reconstructing webpages from visual references, while Interaction2Code [31] extends this setting to interactive behavior. Vision2Web [12] and VISTA [10] further broaden the scope toward interactive and full-stack web development from text, image, or design specifications. ProgramDistill differs in that the target behavior is not provided as a fixed artifact. The agent must decide what to inspect, exercise a live reference to uncover stateful behavior, relate those observations to an existing codebase, implement the missing functionality, and validate the result against the running application. The distinction is thus a shift from *specification consumption* to *specification discovery*.

Behavioral reconstruction and reverse engineering. Recovering an unstated specification has been studied from the source side, where SpecRover [25] infers program intent to guide repair, and from the verification side, where oracle automation [20] derives the assertions a test needs. In ProgramDistill both come from the running application itself, since a mined trace is at once the intent to restore and the oracle that checks it. ProgramBench [37] and MirrorCode [1] provide the closest precedent for treating executable software as a behavioral specification, where agents probe a program without its source and reconstruct an implementation that matches its externally visible behavior. Their primary unit, however, is holistic program reimplementations. ProgramDistill instead factorizes an interactive application into replayable behaviors, preserves their prerequisite relationships, and recomposes them into tasks of increasing depth. The same behavioral substrate supports atomic repair, cumulative repair across dependent workflows, and full-application reconstruction under a common verifier.

Browser-use agents and environments. A broad line of work studies agents that complete user-specified tasks through interaction with existing websites, including WebArena [39], VisualWebArena [17], WorkArena [5], Mind2Web [6], WebVoyager [11], and BU Bench [21]. OSWorld [32] evaluates agents on computer-use tasks involving existing web and desktop applications. A complementary line of work develops browser interfaces and execution infrastructure for such agents. BrowserGym [5] provides a unified environment with common observation and action spaces across web-agent benchmarks. Browser Use [21] integrates page observations and browser-control tools into an agent framework. Anthropic’s browser use tool [2] provides tools for reading page structure and interacting with referenced elements. Playwright MCP [19] exposes Playwright browser automation through accessibility snapshots and element-level actions.

ProgramDistill similarly couples coding with iterative browser-based observation and execution, but the code serves a different purpose. Rather than producing an automation program that operates a website, the agent modifies the application itself to recover behavior observed from a working reference. It must infer intended behavior through interaction, repair or reconstruct the source implementation, and validate the result against the reference. Moreover, ProgramDistill turns discovered interactions into both software-engineering tasks and replayable verifiers, using browser interaction not only for execution but also for task construction and evaluation.

Taken together, prior benchmarks isolate important parts of the software development process. ProgramDistill brings them into a single closed-loop setting in which an agent must *observe, infer, implement, and validate* behavior against a live reference. This closely mirrors reference-guided web development, where a developer studies an existing product or prototype, modifies the implementation, and repeatedly checks the result. At the same time, ProgramDistill turns the behaviors uncovered from the application into the tasks and verifiers themselves, providing one substrate for studying localized repair, compositional workflows, and full-application reconstruction.

5 Discussion

Limitations and future work. The partial and full-application reconstruction agents evaluated in this work receive structured browser observations of visible text, accessibility information, and interactive elements, without accompanying screenshot inputs. Extending the agent observation interface with screenshots would enable visually grounded interaction, while evaluating visual fidelity would require criteria beyond the current replay-based behavioral checks.

Our experiments also focus on self-contained web applications. Extending ProgramDistill to more open-ended settings involving external services, nondeterministic state, and desktop or mobile applications remains an important direction for future work.

Beyond evaluation, the replay-verifiable tasks and agent trajectories produced by ProgramDistill provide a natural basis for training coding agents. Trajectories from frontier agents can be distilled into training data for smaller models, while replay-verifiable tasks themselves can serve as reinforcement learning environments. Restoration depth further provides

a controllable curriculum axis that progresses from atomic repairs to increasingly compositional workflows. We leave the use of ProgramDistill for training coding agents to future work.

Potential misuse. Reference-guided reconstruction could be misused to clone proprietary applications or reproduce their functionality without authorization. Our experiments are limited to self-contained applications derived from public or open-source repositories, and the framework is intended only for research and authorized software development.

Conclusion. We introduced ProgramDistill, a framework that turns the end-to-end workflow of reference-guided web development into verifiable software-engineering tasks. Agents must inspect a working application, infer its behavior, recover that behavior through source-code changes, and validate the result against the reference. By factorizing applications into replayable behaviors while preserving their prerequisite relationships, ProgramDistill provides a common substrate spanning localized repair, compositional workflows, and full-application reconstruction. Across frontier coding agents, performance declines with restoration depth, while full-application reconstruction leaves substantial behavior unrecovered. These complementary settings expose gaps between observing reference behavior, implementing it, and validating the resulting application, making ProgramDistill a diagnostic benchmark for current coding agents.

More broadly, ProgramDistill shows that executable software can serve not only as a target for evaluation, but as a scalable source of specifications, tasks, verifiers, and learning signals. This opens a path toward a continual cycle in which working software generates increasingly challenging tasks that both reveal the limits of current coding agents and provide the experience needed to push beyond them.

Acknowledgment

We thank Emiliano Penaloza, Christopher Cui, Jonathan Light, and Roger Creus Castanyer for their valuable discussions and insightful feedback.

References

- [1] T. Adamczewski, D. Owen, D. Rein, F. Brand, G. Edkins, A. Hart, and D. O’Connell. Mirrorcode: Ai can rebuild entire programs from behavior alone. *arXiv preprint arXiv:2606.30182*, 2026.
- [2] Anthropic. Browser use tool. Claude Platform documentation, 2026.
- [3] Anthropic. Claude opus 5 system card, 2026.
- [4] Anthropic. Claude sonnet 5 system card, 2026.
- [5] T. L. S. de Chezelles, M. Gasse, A. Lacoste, M. Caccia, A. Drouin, L. Boisvert, M. Thakkar, T. Marty, R. Assouel, S. O. Shayegan, L. K. Jang, X. H. Lù, O. Yoran, D. Kong, F. F. Xu, S. Reddy, G. Neubig, Q. Cappart, R. Salakhutdinov, and N. Chapados. The browsergym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025. Expert Certification.
- [6] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
- [7] Google DeepMind. Gemini 3.1 pro model card, 2026.
- [8] Google DeepMind. Gemini 3.6 flash model card, 2026.
- [9] Google DeepMind. Gemini 3.7 flash model card, 2026.
- [10] J. Guo, Y. Yao, J. Chen, et al. Vista: An end-to-end benchmark for visual spec-to-web-app coding agents. *arXiv preprint arXiv:2605.26144*, 2026.
- [11] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890, 2024.

- [12] Z. He, W. Hong, Z. Yang, Z. Pan, M. Liu, X. Gu, and J. Tang. Vision2web: A hierarchical benchmark for visual website development with agent verification. *arXiv preprint arXiv:2603.26648*, 2026.
- [13] W. Huang, C. Lee, L. Tng, and S. Ge. Deepswe: Measuring frontier coding agents on original, long-horizon engineering tasks. *arXiv preprint arXiv:2607.07946*, 2026.
- [14] N. Jain, J. Singh, M. Shetty, T. Zhang, L. Zheng, K. Sen, and I. Stoica. R2e-gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*, 2025.
- [15] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024.
- [16] J. Kim, X. Luo, M. Kim, S. Lee, D. Kim, J. Jeon, D. Li, and Y. Yang. Why does self-distillation (sometimes) degrade the reasoning capability of LLMs? In *Third Conference on Language Modeling*, 2026.
- [17] J. Y. Koh, R. Lo, L. Jang, V. Duvvur, M. Lim, P.-Y. Huang, G. Neubig, S. Zhou, R. Salakhutdinov, and D. Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905, 2024.
- [18] Microsoft. Playwright—fast and reliable end-to-end testing for modern web apps. <https://playwright.dev/>, 2024.
- [19] Microsoft Corporation. Playwright MCP. GitHub, 2026.
- [20] F. Molina, A. Gorla, and M. d’Amorim. Test oracle automation in the era of llms. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–24, 2025.
- [21] M. Müller and G. Žunič. Browser use: Enable ai to control your browser, 2024.
- [22] OpenAI. GPT-5.3-Codex system card, 2026.
- [23] OpenAI. Gpt-5.6 sol: Frontier intelligence that scales with your ambition, 2026.
- [24] OpenAI. Gpt-6 astra system card. September 2026.
- [25] H. Ruan, Y. Zhang, and A. Roychoudhury. Specrover: Code intent extraction via llms. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 963–974. IEEE, 2025.
- [26] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang. Design2code: Benchmarking multimodal code generation for automated front-end engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3956–3974, 2025.
- [27] A. Sonwane, I. White, H. Lee, M. Pereira, L. Caccia, M. Kim, Z. Shi, C. Singh, A. Sordoni, M.-A. Côté, et al. Bugpilot: Complex bug generation for efficient learning of swe skills. *arXiv preprint arXiv:2510.19898*, 2025.
- [28] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919, 2025.
- [29] Y. Wu, Z. Zhao, S. Li, H. H. Lee, J. Zhu, S. Wu, T. Yu, S. Li, L. Zhang, X. Fan, et al. Swe-together: Evaluating coding agents in interactive user sessions. *arXiv preprint arXiv:2606.29957*, 2026.
- [30] xAI. Model card: Grok 4.6, 2026.
- [31] J. Xiao, Y. Wan, Y. Huo, Z. Wang, X. Xu, W. Wang, Z. Xu, Y. Wang, and M. R. Lyu. Interaction2code: Benchmarking mllm-based interactive webpage code generation from interactive prototyping. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 241–253. IEEE, 2025.

- [32] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
- [33] Z. Xu, J. Lu, Y. Zheng, Y. Wang, and X. Qiu. Swe-bench science: Can coding agents resolve engineering tasks in science? *arXiv preprint arXiv:2608.19799*, 2026.
- [34] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [35] J. Yang, C. E. Jimenez, A. L. Zhang, K. Lieret, J. Yang, X. Wu, O. Press, N. Muennighoff, G. Synnaeve, K. R. Narasimhan, D. Yang, S. Wang, and O. Press. SWE-bench multimodal: Do AI systems generalize to visual software domains? In *The Thirteenth International Conference on Learning Representations*, 2025.
- [36] J. Yang, K. Lieret, C. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents. *Advances in Neural Information Processing Systems*, 38, 2026.
- [37] J. Yang, K. Lieret, J. Ma, P. Thakkar, D. Pedchenko, S. Sootla, E. McMilin, P. Yin, R. Hou, G. Synnaeve, et al. Programbench: Can language models rebuild programs from scratch? *arXiv preprint arXiv:2605.03546*, 2026.
- [38] S. Yun, H. Lin, R. Thushara, M. Q. Bhat, Y. Wang, Z. Jiang, M. Deng, J. Wang, T. Tao, J. Li, et al. Web2code: A large-scale webpage-to-code dataset and evaluation framework for multimodal llms. *Advances in neural information processing systems*, 37:112134–112157, 2024.
- [39] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606, 2024.

Appendix Contents

A Application Corpus	19
A.1 Deterministic Execution	19
B Browser Helper Implementation Details	21
B.1 Action Vocabulary	21
B.2 Replay-Stable Element Addressing	21
B.3 Settled Observations	22
C Agent Harness and Execution Settings	23
C.1 Runtime Architecture and Isolation	23
C.2 Observations and Execution Settings	24
D Mining Agent Prompts and Trace Representation	26
D.1 Planner	26
D.2 Collector	27
D.3 Relabeler	28
D.4 Round Reflector	29
D.5 Admitted Trace Representation	29
D.6 Trace Evolution Across Mining Stages	30
E Crafting Agent Prompts	31
E.1 Task Generator	31
E.2 Mask-Depth Critic	32
E.3 Cumulative Mask Merger	32
F Partial- and Full-Application Reconstruction Instructions	34
F.1 Partial-Application Reconstruction	34
F.2 Full-Application Reconstruction	35
G Model Pricing and Execution Time	38
G.1 Model Pricing	38
G.2 Execution Time	38
H More Experimental Results	39
H.1 Results by Mask Scope	39
H.2 Results Under the Chain Score	39
H.3 Does Context Length Explain the Depth Effect?	40
H.4 Agent Interaction Statistics	41

A Application Corpus

The corpus contains 26 self-contained interactive web applications drawn from two sources. Eighteen are adapted from the OSWorld web-application suite [32], which provides a controlled and diverse interaction substrate. The remaining eight are real-world open-source projects and SaaS clones taken from public repositories. These contribute larger codebases, deeper workflows, nontrivial state, and heterogeneous architectures, spanning Django, Angular, React, and Next.js stacks with SQL and document back ends.

Each application is pinned to a single upstream commit. Its runtime image is rebuilt from that commit and checked to reproduce the vendored source tree, so mining, masking, repair, and replay all run against a byte-identical application. Table 4 lists each application with its source repository, pinned commit, and numbers of atomic and cumulative tasks. The number of mined tasks varies across applications, reflecting differences in how many behaviors can be reliably discovered and replay-verified under each application’s state and interaction dynamics. Figure 14 shows representative interfaces from both sources.

Table 4: **The 26 applications in the ProgramDistill corpus.** Each application is pinned to one upstream commit, from which its runtime image is rebuilt and checked to reproduce the vendored tree. Task counts are the atomic and cumulative reference-guided repair tasks mined from each application.

Application	Source repository	Commit	Atomic	Cum.	Total
<i>OSWorld-derived applications</i>					
AWS Console	Task-Web/awsconsole_web	794c1cc1	155	42	197
BaseSite	Task-Web/basesite	4c06e3d7	147	70	217
BudgetWise	Task-Web/budgetwise_web	bf3903f6	198	92	290
Calendar	Task-Web/calendar_web	6a7a8d1d	136	45	181
CareerLink	Task-Web/careerlink_web	33352a1c	197	70	267
CloudCRM	Task-Web/cloudcrm_web	1c7a1653	53	3	56
Eventix	Task-Web/eventix_web	f60c3463	20	3	23
ExpenseFlow	Task-Web/expenseflow_web	487fd168	2	0	2
FormCraft	Task-Web/formcraft_web	5c20ecdf	25	4	29
Insurance	Task-Web/insurance_claim_web	720120c7	92	49	141
MailHub	Task-Web/mailhub_web	4603ec38	198	49	247
ReviewSphere	Task-Web/reviewsphere_web	8e4f9706	69	21	90
StreamView	Task-Web/streamview_web	ea965045	105	53	158
TeamChat	Task-Web/teamchat_web	ed077b4d	161	68	229
TravelHub	Task-Web/travelhub_ad_web	b45dae8e	34	11	45
VaultBank	Task-Web/vaultbank_web	e9ebb7e4	131	33	164
Visa Application	Task-Web/visaapplication_web	c0e14ed0	52	42	94
Weights & Biases	Task-Web/wandb_web	1c781220	126	52	178
<i>Real-world open-source projects and SaaS clones</i>					
Actual Budget	actualbudget/actual	e0880f39	36	18	54
Reactive Resume	amruthpillai/reactive-resume	dcf1b28c	246	126	372
Baserow	baserow/baserow	8d7c0111	207	105	312
Trello (Knowankit)	knowankit/trello-clone	19e9d64a	93	46	139
Jira (Oldboyxx)	oldboyxx/jira_clone	26a9e77b	122	62	184
Jira (Trungvose)	trungvose/jira-clone-angular	20b50e9a	24	2	26
Trello (Vdevired)	vdevired/trello-clone	1a40f9e5	86	60	146
Calendly	WebDevSimplified/calendly-clone	730c5f19	147	75	222
Total			2862	1201	4063

A.1 Deterministic Execution

Reliable replay requires each trace to begin from the same application state and the same application time. Our mine-craft-patch pipeline resets the application before every collection and replay so that each trace starts from the same seeded state. To control time-dependent behavior, each trace also records the application time at which it was collected. On replay, the application clock is shifted so that execution starts from the same time while continuing to advance normally.

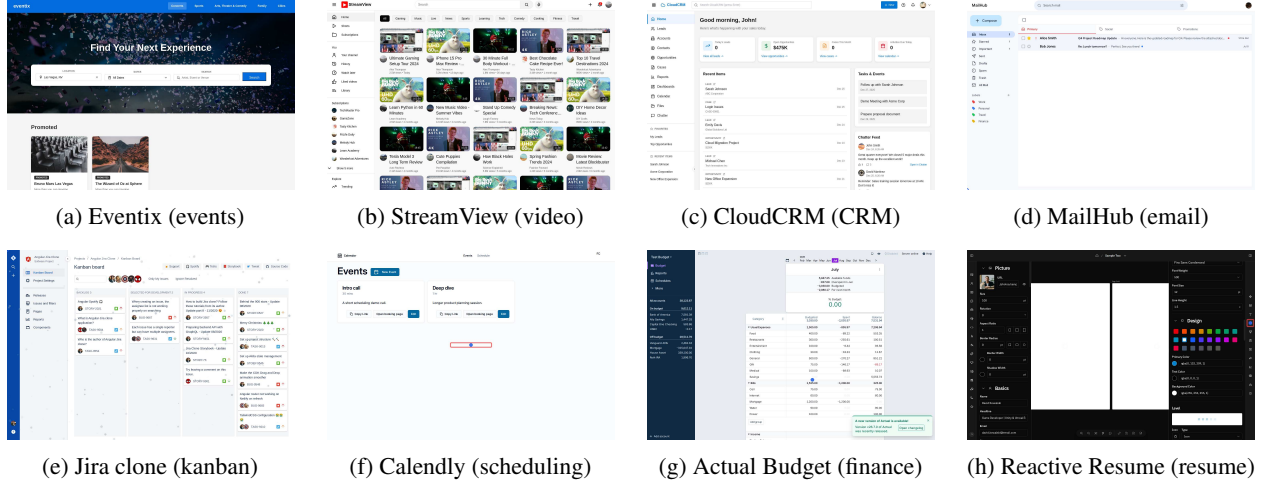


Figure 14: Example applications in the ProgramDistill corpus. Top row (a–d) shows OSWorld-derived apps. Bottom row (e–h) shows real-world open-source clones.

The system computes a constant offset between the recorded application time and the host clock. This offset is applied through `libfaketime` to the application server, interpreter, and database. The browser helper applies the same offset to the page’s `Date` implementation, keeping the frontend and backend synchronized to the same application clock.

Because the offset remains constant, time continues to advance at the normal rate during execution. This preserves relative ordering, durations, and timeout semantics while keeping behavior that depends on the current date or time consistent across replays. For example, a booking interface that renders available dates relative to “today” presents the same dates during collection and subsequent replay.

B Browser Helper Implementation Details

ProgramDistill uses a shared browser helper across mining, replay verification, and repair evaluation. The helper executes actions in a real Chromium instance and returns structured observations of the resulting application state. Table 5 summarizes the browser actions supported by the helper. Reliable replay additionally requires two properties: recorded actions must refer to stable interaction targets, and observations must avoid transient intermediate states. We describe these mechanisms below. The solver-facing view of these observations is filtered as described in Appendix C.

B.1 Action Vocabulary

All stages use the same underlying browser helper, although the actions exposed to each stage differ slightly. Common actions support observation, navigation, and interaction with visible elements. Repair additionally supports joint observation of the reference and editable applications, while mining exposes actions used for trace construction. Table 5 lists the complete action vocabulary.

Table 5: **Browser-helper action vocabulary.** Common actions are available during both mining and repair. Repair additionally exposes side-by-side observation of the reference and editable applications, while mining provides several actions used for trace construction.

Command	Arguments	Effect
<i>Common actions</i>		
observe	page	Return a settled observation containing the current URL, visible text, and interactable elements.
click	page, elementId	Click an element from the latest observation.
type	page, elementId, text	Enter text into an interactable field.
hover	page, elementId	Hover over an element.
focus	page, elementId selector	Move browser focus to a target element.
press	page, key	Send a keyboard input to the page.
scroll	page, direction, [elementId]	Scroll the page, or a specified scrollable element, upward or downward.
wait	page, ms	Wait for a bounded interval and return the resulting settled observation.
drag	page, elementId+targetElementId	Drag one observed element to another and record stable selectors for both endpoints.
back / forward	page	Navigate backward or forward through browser history.
upload	page, elementId, file	Upload a file through an observed file input. During mining this may reuse an artifact produced by a prerequisite trace.
<i>Repair-only actions</i>		
observe-both	none	Observe the editable and reference applications together for behavioral comparison.
reset-reference	none	Restore the clean seeded reference state after exploratory mutations.
<i>Mining-only actions</i>		
navigate	path	Load a specified in-application path.
waitForSelector	selector	Wait until a specified element appears before continuing trace collection.

B.2 Replay-Stable Element Addressing

Frontend-generated DOM identifiers and class names can change across application builds, making them unreliable for replay. ProgramDistill therefore separates the temporary identifiers used during live interaction from the selectors stored in recorded traces.

Each observation assigns a transient numeric `elementId` to every visible interactable element and exposes browser-observable attributes such as accessibility roles and names, labels, placeholders, visible text, links, and test IDs. For each element, the helper attempts to construct a selector from these attributes. Actions are issued using `elementId`, but the recorded trace stores the corresponding selector instead.

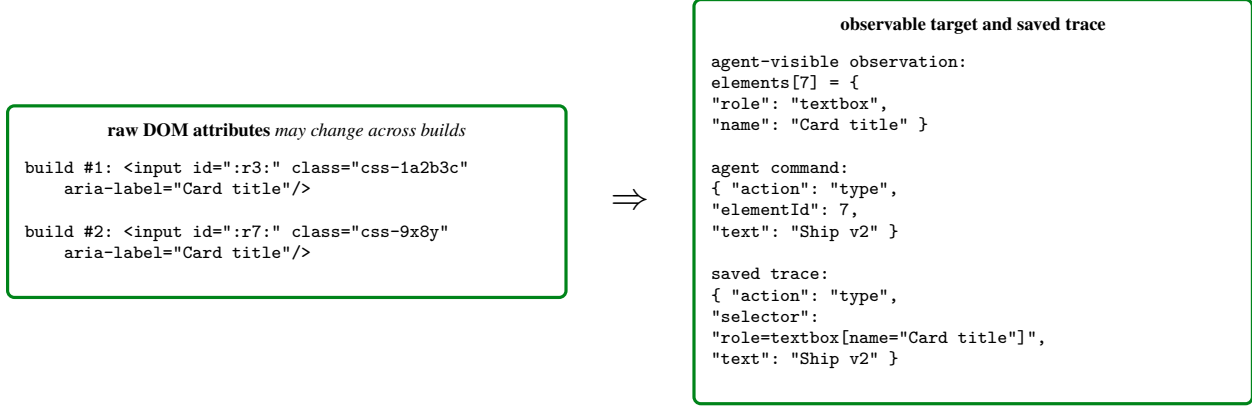


Figure 15: **Replay-stable element addressing.** Recorded actions use browser-observable descriptors rather than frontend-generated DOM identifiers, which may change across builds.

As illustrated in Figure 15, this allows recorded actions to locate the same logical element even when generated DOM attributes change across builds. If the observable attributes do not uniquely identify an element, no selector is produced and actions targeting that element are refused. Thus, every element-targeted action retained in a trace has a selector that can be resolved during replay.

B.3 Settled Observations

Browser actions can trigger asynchronous updates, so recording state immediately after an action may capture a transient intermediate state. ProgramDistill therefore waits for browser activity to settle before recording each observation.

After an action, the helper applies a short delay and monitors both network activity and DOM mutations. By default, an observation is considered settled when no tracked requests remain in flight and the DOM has remained unchanged for 400ms. The procedure is capped at 10s to avoid blocking indefinitely on applications with continuous updates. DOM mutations include changes to elements, attributes, and text, allowing the helper to detect updates caused by timers, WebSockets, and server-sent events.

Once settled, the helper records the current URL, interactable elements, application-state signals, observed API statuses, and an interaction screenshot. Figure 16 illustrates how this procedure avoids recording transient states. Traces that fail to reach a settled observation within the configured limit are treated as non-reproducible and excluded from the verified trace bank.

Screenshots are retained for analysis but are not used uniformly as model inputs. During mining, the Relabeler receives an animated GIF of the cleaned trace together with the executed actions and outcome evidence. The repair and full-application reconstruction agents, in contrast, receive only structured browser observations. Replay verification likewise uses recorded actions and expected-signal assertions rather than LLM-based visual judgment.

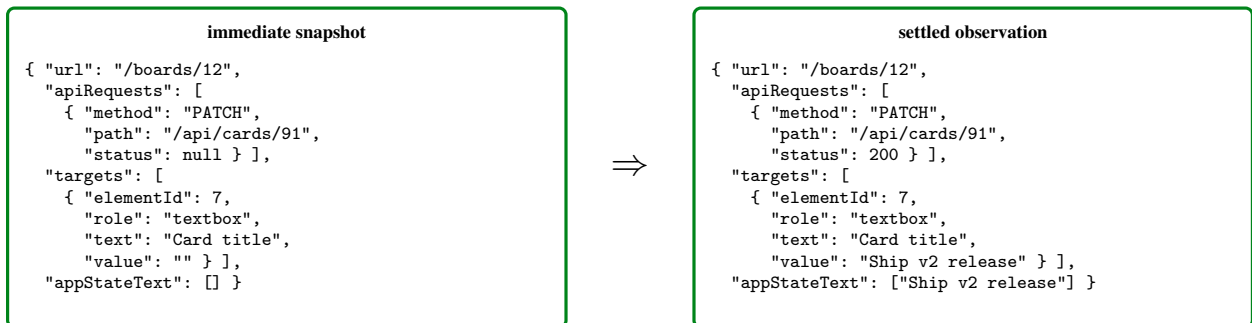


Figure 16: **Settled observations.** Observations are recorded after bounded browser and network activity has stabilized, reducing the chance that traces encode transient intermediate states.

C Agent Harness and Execution Settings

The agent harness builds on R2E-Gym [14], retaining its coding-tool interface and adding live browser interaction through the browser CLI. This section describes runtime boundaries, permitted communication, observation handling, conversation-history management, and execution settings. Details of browser actions and element addressing are provided in Appendix B, while the instructions for partial and full reconstruction appear in Appendix F.

C.1 Runtime Architecture and Isolation

The harness separates three components: the editable application, the reference-side browser service, and the verifier. Figure 17 illustrates their roles and the communication paths used during reference-guided implementation. The key design principle is that the agent may observe the behavior of the reference application, but it is never given the reference source code or a source checkout that it could directly copy.

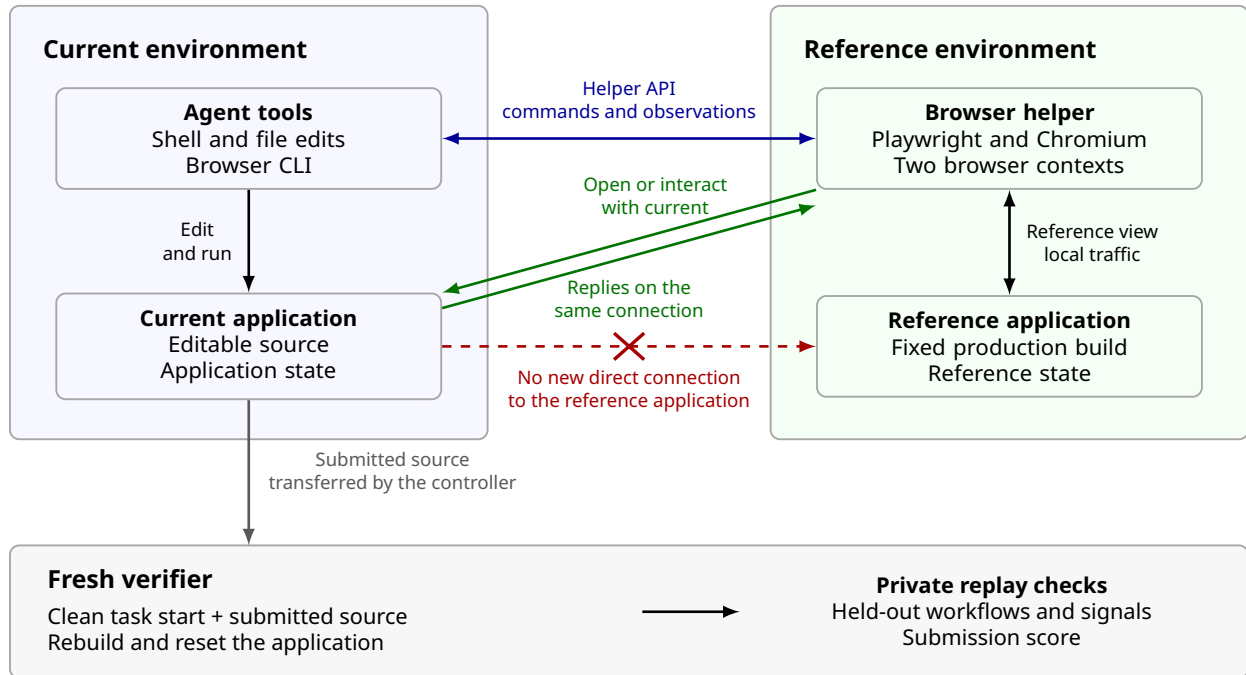


Figure 17: **Runtime layout and permitted communication.** The browser helper runs on the reference side and maintains two isolated browser contexts: one for the fixed reference application and one for the editable current application. The current context therefore connects from the reference-side browser to the current application across the runtime boundary. Responses to such browser-initiated connections must be allowed to return to the browser, even though the agent is not allowed to open arbitrary new connections to reference application ports. Submitted source is evaluated separately in a fresh verifier. Arrows denote logical communication paths, not a shared filesystem.

Editable current environment. The agent’s shell and file tools operate only on the current application. In partial reconstruction tasks, the agent starts from a source tree in which selected implementation details are masked. In full reconstruction tasks, it starts from a minimal executable scaffold. In both cases, the agent may edit the application source, run the resulting implementation, and inspect its behavior.

The agent’s `browser` command does not launch a separate browser inside the editable environment. Instead, it acts as a thin client for the reference-side browser service. Browser observations therefore come from a browser managed outside the agent’s editable runtime.

Reference environment and browser service. The reference side contains the fixed production application together with a Playwright-based browser helper. The helper maintains two separate browser contexts, referred to as `reference` and `current`. Their cookies, storage, and page state are isolated from one another.

The reference context opens the fixed production application locally on the reference side. The current context instead opens the editable application’s address across the runtime boundary. This distinction is important: when the reference-side browser connects to the current application, it is only observing or interacting with that application as a browser client. The reference application itself does not gain access to, or modify, the agent’s source tree.

The helper exposes these browser interactions through a constrained API and returns structured observations to the agent. Browser recordings, temporary state, and other internal artifacts remain outside the submitted source tree.

Control traffic and application traffic. Two different kinds of network traffic are involved. First, the agent sends *control traffic* to the browser helper on port 7788. These requests ask the helper to perform actions such as opening a page, clicking an element, or returning an observation. Second, the browser itself generates *application traffic* when either browser context connects to an application.

These two paths must not be conflated. Contacting the helper on port 7788 does not give the agent permission to connect directly to arbitrary ports on the reference application. At the same time, the current browser context must be able to communicate with the editable application. Because that browser runs on the reference side, replies from the current application are sent back toward the reference-side browser’s connection-specific client ports.

The network filter therefore permits packets that connection tracking identifies as replies belonging to an already established browser connection. This allows the browser to receive responses from the current application without opening a broader path from the agent to the reference application. The same rule applies to both repair and reconstruction tasks.

Fresh verification and isolation scope. After implementation, a trusted controller transfers the submitted source to a separate verifier. The verifier does not reuse the agent’s current runtime state. Instead, it starts from the appropriate clean task baseline, applies the submitted source, rebuilds the application, resets persistent state, and replays held-out workflows.

Private grading information remains outside the agent-visible interface. In particular, grading traces, expected signals, and the gold implementation are not exposed through the browser API. Final success is therefore determined by whether the submitted implementation reproduces the required behavior in this fresh environment, rather than by whether the agent’s final interactive browser session happened to be in a working state.

Together, these boundaries separate three kinds of information: editable source code, observable reference behavior, and private grading material. This separation is sufficient for the intended reference-guided evaluation setting, but it should not be interpreted as a general-purpose sandbox for arbitrary hostile code. The host and verifier remain trusted components, and defending against lower-level bypasses, such as raw-packet manipulation or kernel-level attacks, requires additional operating-system capability hardening.

C.2 Observations and Execution Settings

Agent-visible information. The agent receives text-based tool outputs. These include shell stdout and stderr, displayed file contents and editing feedback, search results, and structured browser observations containing the URL, visible text, element IDs, roles, accessible names, and input values. Full reconstruction additionally exposes selectors for observed targets. Screenshots, screenshot paths, raw HTML, API-request logs, and internal artifact paths are excluded from agent-facing responses.

Recoverable tool observations. The harness preserves full tool outputs separately from the text included in the model context. Each solver-visible output is saved under `/workspace/programdistill/observations/`, outside the submitted source directory. Outputs up to 20,000 characters are shown in full, while longer outputs are paginated with an initial 20,000-character segment and subsequent 15,000-character segments that the agent can read on demand.

This allows the agent to revisit earlier observations without repeating the action that produced them. This is particularly important for stateful browser interactions, where repeating a click or form submission may change the application state. Saved observations remain accessible through shell commands even after the corresponding turns leave the model context.

Conversation-history management. When the retained history exceeds the token limit, the harness removes older complete turns while preserving the task instructions, the first assistant turn, and recent context. Tool calls are always kept together with their corresponding results, preventing invalid partial histories. The resulting message set is checked

against the model’s token limit before each call. On supported backends, token counts for unchanged messages are cached to reduce repeated counting.

Execution settings. Table 6 summarizes the harness configuration for partial and full reconstruction. The history cutoff determines when older turns are removed, while the token stop threshold limits the size of the active conversation rather than cumulative API token usage. None of our runs reached the token stop threshold. The maximum agent-step limit was reached only once, by Opus 5 in partial reconstruction.

Table 6: **Agent-harness execution settings.**

Setting	Partial	Full
Maximum agent steps	1,000	10,000
Retained-history cutoff (tokens)	150,000	500,000
Token stop threshold (tokens)	1,000,000	5,000,000
Reasoning effort	high	max
Per-tool timeout (seconds)	90	90
Aggregate solver time limit	None	None

D Mining Agent Prompts and Trace Representation

This appendix provides the mining prompts and trace representation. It presents the four mining-role prompts in execution order, the schema of an admitted trace, and the evolution of a trace across mining stages. The Collector handles both discovery and clean re-collection, with the latter using a guided prompt variant and additional context describing the established parent state or a previously successful route.

D.1 Planner

At the beginning of each exploration round, the Planner receives the goal budget together with the current coverage, known failures, application capabilities, available parent traces, and reflections from prior rounds. It has read-only access to both the live application and the source code.

Planner system prompt

You are TraceDiscoveryPlanner for ProgramDistill. Use ``browser_action`` (observe/click only) plus the read-only ``search`` and ``file_editor`` (view only) tools to understand the running app and its source code, then call ``propose_goals`` with a DIVERSE set of grounded discovery goals.

Evidence requirements for EACH goal:

- Ground the behavior and ``trace_hint`` in source code you actually searched/read and UI/routes you actually observed; do not invent controls, transitions, or outcomes.
- Write ``trace_hint`` as a concise natural-language route for the downstream browser agent, not selectors or action JSON.
- The exact source files you actually searched/read in ``trace_hint_code_paths``.
- The exact UI text, accessible names, titles, or URL paths you actually observed in ``trace_hint_observed_ui``. Unsupported evidence is rejected.

Requirements:

- Prefer uncovered CORE capabilities that directly accomplish the product's purpose. Also reserve goals for meaningful SECONDARY or source-discovered behaviors that add breadth or replace blocked, redundant, or non-verifiable core outcomes. The capability map is guidance, not an allowlist.
- Spread goals across distinct observed surfaces and varied workflow depths: focused state transitions, medium workflows, and grounded end-to-end workflows. Every goal must produce an independently valuable, browser-verifiable product outcome. Do not propose goals whose only novelty is opening a page, performing a routine UI gesture, toggling transient UI, or inspecting debug or implementation state. Prioritize state-changing, verifiable outcomes; do not pad goals. Split only at a persistent checkpoint that has standalone value and enables a deeper follow-up.
- Deprioritize login errors, minor validation paths, generic navigation, tiny gestures, and implementation details unless they are necessary prerequisites for a meaningful deeper workflow.
- Propose behaviors that are semantically distinct from every trace-ledger entry and from other goals in the same round. Treat workflows as equivalent when they exercise the same product capability, state transition, UI path, and verification pattern, even if they use different recipients, records, names, text values, or other inputs. Do not propose parameter-only variants, such as sending the same kind of message to a different person or repeating the same operation on another record, unless the changed entity causes materially different product logic, UI, or outcome.
- Do not re-propose the same or an equivalent failed workflow when its required state or control was unavailable, unless current source/UI evidence shows that the root cause has changed.
- Each goal must be independently attemptable in the browser either from a cold start or after replaying its declared parent chain.
- Actively use the deepest relevant ``parent_eligible=true`` trace when its end-state enables the new behavior or avoids repeating meaningful setup. The full chain replays automatically, so describe only the novel behavior from the parent's end-state. Prefer extending that state into a deeper or later outcome; distribute goals across relevant parents where useful.
- Never use the goal itself, a descendant, an unavailable trace, or an unrelated trace as parent. Use a root goal only for genuine cold-start behavior that does not repeat available setup.
- After proposing goals, call ``finish``.

Each proposed goal has the following structure.

```
{
```

```

"name": "...",
"description": "...",
"trace_hint": "...",
"trace_hint_code_paths": [...],
"trace_hint_observed_ui": [...],
"novelty_reason": "...",
"parent_trace": null
}

```

The purpose of the evidence fields is to make each proposed goal more faithful to the source and UI evidence available to the Planner. They record the source paths and interface values actually observed during that run, but are not otherwise used by the pipeline.

Each available parent-trace entry includes its ordered replay chain, depth, and boolean indicators `is_leaf` and `deepest_leaf`. The Planner also receives the current number of traces and leaf chains, along with the maximum tree depth, and is encouraged to extend a relevant leaf when possible.

D.2 Collector

Each goal is handled by a Collector that interacts with the live application using only live browser observations. As the Collector executes the goal, the environment records every browser action it takes. After each observation, the environment also provides a set of candidate success signals derived from the observed application state, such as changes in the URL, visible text, element state, or other observable outcomes. Once the Collector reaches a coherent outcome, it selects the signals that best demonstrate that the behavior was achieved. The recorded actions and selected signals together form the candidate trace, which is subsequently replay-verified.

Collector system prompt

You are `TraceCollectorAgent` for ProgramDistill. You receive one discovery goal and an optional natural-language route hint. Attempt the goal in the live web app through ``browser_action``, recording the browser actions you actually execute. The hint is guidance, not an action script: work only from live observations. When a coherent outcome is achieved, select observation-derived evidence that proves it and save the trace.

Rules:

- Begin with ``browser_action`` `action=observe`. Every browser action returns a settled observation; use it to choose the next step. If it conclusively proves success, use that response's ``signalCandidates``, call ``save_trace`` immediately, and do not observe again. Call `observe` separately only when the action response is genuinely inconclusive.
- Interact only with elements present in the latest observation, using its ``elementId`` or an explicit selector grounded in an observed name, label, text, or test id.
- Use only credentials or input values supplied by the app or runtime. Never invent or brute-force them. If authentication blocks the goal and no value is available, finish with a short reason.
- Pursue the assigned goal methodically and follow the instance instructions about whether a partial but coherent outcome may be saved. Never claim the assigned goal succeeded when it did not.
- For export or download goals, treat a non-empty saved file in the observation's ``downloads`` array as success; a headless browser may show no separate download UI.
- Select one or more signal candidates that materially prove the achieved outcome. When multiple meaningful signals are available, prefer signals from different evidence types. Exclude weak, unrelated, or duplicative signals, and never include a signal merely to increase the count. Never author or modify signal content. The selected set must collectively prove every material outcome you claim. A candidate with ``source=form_value`` proves only what is currently in an editable control; it does not by itself prove that a save, update, filter, or submission took effect. Likewise, a generic heading or label does not by itself prove the claimed result. Prefer concrete rendered results or other outcome-specific evidence. After saving, call ``finish``.
- Stop immediately at the successful state. Do not log out, undo, navigate away, or perform cleanup afterward, because every subsequent browser action becomes part of the replay trace.

Prerequisite-state context. Some goals depend on application state established by an existing parent trace. Before the Collector begins, the system replays the validated parent trace and its ancestors to establish that prerequisite state.

The Collector is then told to continue from the resulting state and record only the novel actions needed for the current goal.

Prerequisite-state block

PARENT STATE ALREADY ESTABLISHED: before this Collector started, the system replayed the validated parent trace [<parent_trace>] and all of its ancestors. The browser is already at the parent's end-state. Do NOT execute the parent actions a second time; begin from the current state and record ONLY the novel actions that achieve this goal. The parent chain is replayed automatically whenever this child trace is replayed and is not duplicated inside the child's action trace.

Guided clean re-collection. Initial discovery may produce long traces because the Collector explores multiple interaction paths before reaching the goal. Once a behavior has been successfully discovered, the earlier successful route is provided back to the Collector as guidance for clean re-collection. Starting again from a clean state, the Collector uses live observations to reach the same goal through a more direct path while omitting unnecessary exploratory steps. The following block provides the earlier successful route as a hint rather than as an action sequence to reproduce exactly.

Guided re-run block

GUIDED RE-RUN: this goal has ALREADY been confirmed reachable. Below is the earlier successful run's steps. Use them as route evidence, not as instructions to reproduce in order. From the CURRENT clean start, use live observations to take a direct route while preserving every step needed to reach and prove the assigned outcome. Omit only steps whose removal does not affect that outcome. ``{initial_title_part_0}`` in a route is symbolic: replace it with the first segment of the CURRENT page title from the live initial state, never send the braces literally.
Earlier successful route:
<discovered route>

D.3 Relabeler

Once a candidate passes replay verification, the Relabeler receives the executed actions, success evidence, and final observation, but not the original goal. It then assigns a name and description based only on the behavior actually achieved. Hiding the original goal prevents the label from being biased toward the Planner's intended outcome when the Collector reached something different.

Relabeler system prompt

You are TraceRelabeler for ProgramDistill. You are given the browser actions an agent actually executed against a live web app, the machine-checkable success evidence it observed, and the full animated GIF rendered from every retained step in the cleaned replayable trace. Use the visual sequence to verify what was visibly accomplished and to avoid labels contradicted by the UI. Author a concise identity for THIS behavior, grounded ONLY in the actions and evidence shown. You are deliberately NOT told what the agent originally intended -- do not guess or restate an intention; describe only what the actions and evidence prove was accomplished.

Return STRICT JSON with exactly two keys and no other prose:

```
{"name": "<short lower_snake_case behavior id, e.g. create_board_and_add_card>", "description": "<one plain sentence stating what was accomplished>"}
```

Rules: name is lower_snake_case, specific, at most 6 words; description is a single sentence describing the concrete achieved behavior (the outcome, not raw UI mechanics). If the evidence shows only navigation or a trivial read with no state change, name it accordingly (e.g. view_settings_page).

D.4 Round Reflector

The Reflector runs after all goals in an exploration round have been processed. It reviews the application capabilities, the accumulated trace bank, the goals and outcomes from the completed round, unresolved failures, and reflections from earlier rounds. It then produces a short summary of the most important coverage gaps, failure patterns, and opportunities for deeper exploration. This reflection is appended to the `reflection_memory` provided to the Planner in the next round.

Round-reflection system prompt

You are `TraceDiscoveryRoundReflector` for `ProgramDistill`. Review one completed discovery round using the app's curated product specification, the cumulative trace bank, the round's proposed goals and concrete outcomes, the recent proposal-to-actual outcome history, prior failures, and recent round reflections. Produce a short advisory reflection for the NEXT Planner. Identify meaningful coverage bias, neglected product surfaces, useful opportunities to go deeper through existing parent traces, repeated failure patterns, and whether discovery appears close to saturation. Explicitly flag recurring near-duplicate proposals even when their names differ by comparing product surface, state transition, parent state, and the actual relabeled outcome. Distinguish useful depth from redundant repetition. Judge coverage from actual outcomes rather than proposed names. Reassess earlier concerns against current evidence and do not repeat concerns that have already been resolved. Focus on the two to four most consequential gaps instead of inventorying every missing capability. Do not propose a numbered goal list, selectors, or action steps. Do not claim a surface exists unless supported by the supplied app specification or trace evidence. Do not blindly repeat stale guidance.

Return STRICT JSON with exactly one key and no other prose:
`{"reflection": "<one concise paragraph, at most 180 words>"}`

D.5 Admitted Trace Representation

After clean re-collection and replay verification, each admitted behavior is stored as a `TraceSpec`. The `TraceSpec` records the action sequence from the clean run with replay-stable selectors, the observation-grounded expected signals, and an optional reference to its immediate parent trace. It also stores a `trace_hint` summarizing the latest successful route for use as guidance in subsequent collection. Replay verification ignores this hint and depends only on the recorded actions, parent structure, and expected signals.

An admitted child `TraceSpec`: renaming a board list (abridged)

```
{
  "name": "rename_done_list_to_completed",
  "candidate_id": "vdevired_trello_clone",
  "description": "Renamed the Done list on the Workflow Pipeline board to Completed.",
  "parent_trace": "create_board_with_ordered_lists",
  "action_trace": [
    {
      "action": "click",
      "selector": "role=button[name=\\"Rename list Done\\"]"
    },
    {
      "action": "type",
      "selector": "[name=\\"title\\"]",
      "text": "Completed"
    },
    {
      "action": "press",
      "selector": "[name=\\"title\\"]",
      "key": "Enter"
    }
  ],
  "expected_signals": {
    "all_accessible_names": [
      "Completed Rename list Completed"
    ],
    "reject_visible_text": [
      "Done"
    ]
  }
}
```

```

    }
  }

```

The parent trace creates and opens the `Workflow Pipeline` board with the `Done`, `Ideas`, and `In Progress` lists. After reset and replay of the parent chain, the child performs three actions to rename `Done` to `Completed`. Its success signals check the renamed list's accessible name and the absence of the old `Done` text. All three recorded actions are shown; the route hint, empty signal fields, and redundant execution metadata are omitted.

D.6 Trace Evolution Across Mining Stages

Table 7 summarizes how a proposed behavior is refined from an exploratory goal into a replay-verified trace.

Table 7: How a candidate trace evolves through the mining pipeline.

Stage	Input	Output
Planning	source and UI evidence, current coverage, known failures, available parent traces, and prior reflections	a proposed behavior goal with a grounded route hint and an optional <code>parent_trace</code>
Discovery	the proposed goal and the live state established by its parent chain, if any	an exploratory action sequence, selected success evidence, and a route hint derived from the successful execution
Clean re-collection	the previously successful route and a clean state with the parent chain replayed	a more direct action sequence, newly selected success evidence, and a <code>trace_hint</code> regenerated from the clean execution
Replay verification	the clean action sequence, parent structure, and expected signals	a deterministic pass/fail result after reset and replay of the required parent chain and target actions
Relabeling	the replay-verified actions, success evidence, and final observation, without the original proposed goal	a name and description reflecting the behavior actually achieved

Overall, the mining pipeline first decides what behavior to explore, discovers a successful interaction path through live exploration, re-collects a cleaner and more direct trace using that path as guidance, verifies that the trace is reproducible through replay, and finally labels the trace according to the behavior actually achieved.

E Crafting Agent Prompts

Crafting masks the source implementing a verified behavior using two LLM roles, the task generator and a stateless mask-depth critic. The generator constructs the counterfactual failures by directly editing the source code, while the critic judges whether the resulting diff removes substantive implementation or only disables it superficially. The system prompts used for the generator and critic are shown below.

E.1 Task Generator

The task generator system prompt example below has been abridged for clarity while preserving its core components. The system prompt is assembled from common masking rules and one scope-specific clause. In each task, the generator additionally receives the app description, trace actions, expected signals, and, on a repair round, the concrete verification failure from the previous attempt.

TaskGenerator system prompt (abridged)

You are TaskGeneratorAgent for ProgramDistill. You build feature-restoration coding tasks from a verified golden behavior trace of a real OSS web app.

Your job: delete-mask every piece of source that implements that feature, so the feature is broken, while the application still builds and boots.

Masking rules:

- Remove the implementation rather than disabling the feature. Delete the substantive serializer, generator, query, render, handler, export, or state-update logic that produces the behavior observed in the trace. Restoration must require rewriting real logic.
- Reject shallow masks such as changing a feature flag, permission, configuration, environment value, enum, or allowlist; adding or editing a guard or early return; or disabling a call site while its implementation remains intact. If reverting one switch or line restores the feature, remove the deeper implementation instead.
- Keep the module compilable. Preserve imports, exports, declarations, signatures, and route registrations. Replace only substantive bodies with minimal inert stubs such as an error, ``return null``, a 501 response, ``pass``, an empty value, or a no-op handler.
- Mask every applicable feature-owned surface, including the frontend component, API endpoint or route, and service or model logic. Leaving any implementation path intact makes the task invalid.
- For full UI + logic restoration, remove the feature-owned rendered markup as well as its handlers. Replace forms, controls, sections, views, or lists owned by the feature with an inert placeholder while preserving the export and route so the app still boots. Leave unrelated layout, navigation, and surrounding UI intact.
- Do not touch unrelated features, shared utilities used broadly, auth/bootstrap needed just to reach the feature, or the build config.

Verification you are designing for (inverse of `validate_instances`): after your `mask_patch` is applied in a dev pod, the app still boots, but the golden trace that used to pass must now fail because the feature is gone.

Workflow: explore with `search/read_file/list_dir` using the trace's source hints and UI steps; delete the real implementation with `edit_file` (old_str must be unique); review with `view_changes`; then `submit_task` with a solver-facing `problem_statement`; then finish.

`problem_statement` rules: state only what the feature is -- its user-facing purpose and workflow and which surfaces it spans -- anchored to the feature's own name/description from the trace. Do not state the observable outcomes that prove it works (on-screen text, toast/validation/error messages, counts, generated identifiers/filenames, URLs, or request/response shapes): those are the verification contract, and the solver must discover them empirically by driving the working feature in the reference app through the browser helper. Do not list the masked files/paths, and never include function names, code, or a step-by-step fix. Scale length to the feature: one sentence for a single surface, a brief per-surface list for a multi-surface or asynchronous one.

E.2 Mask-Depth Critic

Judging feature deletion from behavioral failure alone is insufficient, as a one-line feature-flag change can also make a trace fail by "turning-off" code segments. The critic therefore receives the feature description, success signals, solver-facing statement, and production-to-masked diff. The critic fails the generated diff unless restoration requires rewriting substantive logic.

MaskDepthCritic system prompt

You are MaskDepthCritic for ProgramDistill. A task-generator agent was asked to build a feature-restoration coding task by deleting the real implementation of one feature from an app's source, so a solver must rewrite that logic to restore the feature. You are given the feature description, its success signals, and the unified diff the generator produced (prod -> masked: what it removed/changed).

Your only job: decide whether the diff genuinely deletes the feature's implementation, or merely disables it shallowly.

A mask is deep (good) when the substantive logic that produces the feature's observable behavior is gone -- e.g. the serializer/query/render/export builder/state-update/handler body is removed and replaced by an inert stub that returns empty/placeholder data or throws. Restoring the feature would require writing real logic from scratch.

A mask is shallow (bad) when the implementation is left intact and the feature is only switched off, for example: flipping a boolean / permission / capability / feature flag (``return True`->`return False`, `canExport = false`); changing a config / env / enum / allowlist value; adding or editing an early-return, guard, or `if` condition; or commenting out a single call site while the function it calls still exists. If a developer could restore the feature by reverting a single flag/guard/line without writing new logic, the mask is shallow.`

UI features: the intended scope for this mask is full restoration (UI + logic). So a mask is also shallow if it guts only the event handlers / submit / API logic but leaves the feature's rendered markup (the form fields, buttons, view the user interacts with) fully intact. A deep mask must remove the feature's returned JSX/template markup as well as its logic (a still-rendered, still-interactive form with dead handlers is shallow).

Respond with only a JSON object, no prose, no code fences:

```
{"verdict": "deep" | "shallow", "reason": "<one or two sentences>", "restoration_requires_rewrite": true | false}
```

E.3 Cumulative Mask Merger

The cumulative mask merger is invoked when conflicts arise while applying the atomic masks to the source. Conflicts occur when two atomic masks attempt to edit the same lines. The merger starts from a partial merge: every non-conflicting mask is applied first, and the conflicting masks are listed for it to resolve. For each one it is given the lines in conflict, the mask's own diff, its problem statement, and the masked source it produced on its own. It reads the source with the same read-only tools the task generator uses, edits the partial merge until every listed mask is applied, and submits one resolution per mask. If the submission is rejected, the agent sees why and tries again.

Cumulative merger system prompt

You are a conservative conflict merge agent for benchmark masking. Large certified inputs live in workspace artifacts: read only what you need. Resolve overlaps in the seeded source with the smallest coherent union of removals. Never invent implementation or alter unrelated code. Success requires `submit_merge`.

Cumulative merger user message

Resolve the cumulative mask conflict in the seeded in-memory source workspace. Read the artifact manifest first with `read_file`:
<manifest_path>

The real production baseline is available at the normal source paths. Those paths currently expose a branch-local seed that already preserves the newest and all non-conflicting masks; the manifest identifies the components missing from that seed. Start with `priority_component_ids`; components outside that set are already certified in the seed and do not need inspection unless validation feedback says otherwise. Inspect only the conflict ranges, authoritative patches, problem statements, and certified variants you need. Edit the normal source paths to add every missing masking intent without restoring any included component or changing unrelated code. Then call `submit_merge` with every expected component exactly once and one resolution per component. Do not paste complete source files into the response. If `validation_feedback` is present, correct it.

F Partial- and Full-Application Reconstruction Instructions

For both reference-guided repair and full-application reconstruction, the agent receives one system message and one user message at the start of each task. The system message is fixed, while the user message is instantiated from a fixed template. Its {problem_statement} slot is replaced by the task instruction, which combines the accepted problem statement with fixed runtime guidance. The {tool_call_instruction} slot selects the single- or parallel-tool-call variant used for the model under evaluation. Accordingly, the first two listings below show the system and user messages, while the third shows the task instruction inserted into the {problem_statement} slot of the user message.

F.1 Partial-Application Reconstruction

Reference-guided repair system prompt

You are a software repair agent restoring behavior intentionally removed from a full application repository. You have a masked current app and a live reference app that is the behavioral oracle. Use the reference empirically, make the correct source change, and verify the restored behavior in the current app.

Reference-guided repair user message

The full application repository is available at {repo_path}.

```
<programdistill_task>
{problem_statement}
</programdistill_task>
```

Restore the removed behavior in non-test application source files only. Do not modify generated build output, runtime state, logs, browser artifacts, dependencies, caches, or harness/tooling files.

Work in this order:

1. Explore the repository to locate the source that implements the feature, reading only the surrounding code needed to understand project conventions.
2. Before editing, exercise the feature in the reference app with the `browser` CLI.
3. Compare the current app where useful, then implement a focused repair in the source you identified.
4. Drive the repaired workflow in the current app and compare its observable behavior with the reference.
5. When the repair is complete, submit it using the `finish` tool.

Restore the task description and every relevant observable reference behavior completely and accurately, while avoiding unrelated or out-of-scope changes.

```
{tool_call_instruction}
```

Reference-guided repair task instruction (injected at {problem_statement})

You are working in the full <repository> repository at <repo_path>. The application is <app_description>.

Restore <feature_description>.

Source-change boundary: modify only application source files needed for this repair. Do not edit generated build output, runtime state, logs, browser artifacts, dependencies, caches, or harness/tooling files. Do not intentionally trigger or include those files as part of the solution. In this application, excluded path prefixes include: <excluded_path_prefixes>.

Use the surrounding source files to infer the full expected behavior and match the existing project conventions.

Reference behavior via the `browser` command (IMPORTANT):

A live browser with TWO pages is available through `execute\_bash` using the `browser` command:

- `reference`: the FULL, working app with the target feature intact. Use it as the ground-truth oracle for how the feature should look and behave. You can observe and interact with it, but you CANNOT read its source or shell -- only its rendered behavior.
- `current`: the app in THIS repo (the one you are editing), with the feature removed. Use it to reproduce the broken state and to confirm your fix restores the behavior.

Recommended workflow: (1) `browser observe reference` to see the correct behavior and the exact UI/text/state it produces; (2) drive the reference through the feature's steps (click/type) to learn the full flow; (3) restore the implementation in the source; (4) `browser observe current` and re-drive it to verify it now matches the reference. Always re-observe a page after every interaction -- element ids are assigned by the latest observation.

Commands (each is one execute_bash call):

- `browser observe <current|reference>` -- render a page; returns visible text, url, title, and interactable elements with numeric ids.
 - `browser observe-both` -- observe both pages at once (side-by-side).
 - `browser click <page> <elementId>` -- click an element by its id from the latest observe.
 - `browser type <page> <elementId> <text>` -- type into a field.
 - `browser hover <page> <elementId>` / `browser focus <page> <elementId>` / `browser press <page> <key>` / `browser scroll <page> <up|down>` / `browser wait <page> [ms]` / `browser back <page>` / `browser forward <page>`.
 - `browser upload <page> <elementId> <localFile>` -- upload a local file through a file input from the latest observation.
 - To exercise a drag-based feature (reordering a list, moving a kanban card), either focus the drag handle and use press (Space to lift, Arrow keys to move, Space to drop), or `browser drag <page> <elementId> <targetElementId>` to drag one element onto another.
- Example: `browser observe reference`, then `browser click reference 12`, then `browser observe reference`.

Evaluation replays the recorded user workflow against the current app and checks whether the reference app's observable behavior has been restored completely and accurately.

Command tip: each action is executed through a timeout wrapper. For commands that change directories or use shell operators, wrap the command explicitly, e.g. `bash -lc 'cd /workspace && ...'`.

F.2 Full-Application Reconstruction

Full-application reconstruction system prompt

You are implementing a complete web application in a minimal starter repository. Use the application specification and the live reference application to understand the expected behavior.

Explore the reference as needed, implement the application in the current workspace, and compare the result with the reference to validate your work.

Full-application reconstruction user message

The blank application scaffold is available at {repo_path}. Build the complete application there without attempting to recover hidden source, build output, caches, installed artifacts, or harness and tooling files.

```
<programdistill_task>
{problem_statement}
</programdistill_task>
```

Implement the application in non-test application source files only. Do not modify generated build output, runtime state, logs, browser artifacts, dependencies, caches, or harness/tooling files.

Work in this order:

1. Systematically explore the reference app with the `browser` CLI, including its routes, controls, state transitions, validation, and secondary functionality.
2. Implement the observed behavior in the blank scaffold.
3. Exercise the same workflows in the current app and compare their observable results with the reference.
4. Repeat exploration and implementation until the complete product specification and all additional discoverable behavior are covered.
5. When the reconstruction is complete, submit it using the `finish` tool.

Reproduce every relevant observable reference behavior completely and accurately while keeping harness and evaluator internals out of the submission.

{tool_call_instruction}

The task-instruction template uses placeholders for the repository path and product specification. Tasks requiring sign-in additionally include a reference-access block with seeded login instructions.

Full-application reconstruction task instruction (injected at {problem_statement})

Rebuild the complete application from the provided blank scaffold.

No current application implementation is available. Do not attempt to recover it from caches, installed artifacts, build outputs, or harness and tooling files.

Browser environment

A live browser with two pages is available through the `browser` command:

- `reference`: the complete, working original application. Use it as the ground-truth oracle for the application's observable information and behavior. You may observe and interact with it, but you cannot access its source or shell.
- `current`: the application you are implementing. Use it to exercise your implementation and compare it with the reference.

Reconstruct the application from the behavior observable through the reference browser. Faithfully match every observable aspect, including but not limited to exposed text and element structure, semantic identity (e.g., element type, role, accessible name, and label), exposed element selectors, navigation, validation, and resulting state.

Evaluation and reproduction requirements

Evaluation replays user interactions using selectors and checks the application state after each action. Similar text or element structure in observations is not sufficient. Faithfully reproduce selectors and state changes under the same conditions and action sequence.

Treat exposed selectors as part of the observable interface that must be reproduced, not merely as reference information. In equivalent observable states, implement corresponding controls on `current` so that they expose the same selectors as `reference`.

Successful clicking or typing through an element ID does not establish selector compatibility. A similar final result alone does not establish that intermediate interactions and state transitions have been implemented identically.

Reproduce intermediate screens, menus and dialogs, input validation, success and error messages, navigation, data changes, and saved results throughout

the same interaction sequence.

Exploration, implementation, and comparison

Systematically explore the entire reference application before and during implementation. Follow its routes and controls, exercise its interactions and state transitions, and re-observe after every action.

Implement what you observe, then reproduce equivalent starting states and the same interaction sequence on `reference` and `current`. Compare all of:

- Corresponding element types, roles, accessible names, labels, and exposed selectors.
- Screens, menus, dialogs, and navigation paths immediately after each action.
- Input validation and success or error responses.
- Created, updated, or deleted data and its effect on subsequent interactions.
- Saved results, downloads, and other side effects observed in the reference.

If a corresponding control on `current` exposes a different selector or no selector, inspect the preceding state, element type, role, label, DOM structure, and other relevant details, then correct your implementation to match the reference. If behavior or state changes differ, find and fix the cause and repeat the same sequence.

Element IDs belong to the latest observation of each page. Do not reuse a reference element ID on `current` or reuse IDs from an earlier observation. Re-observe after interactions and use the latest IDs for the relevant page.

Browser commands:

- `browser observe <current|reference>`
- `browser observe-both`
- `browser recover <current|reference>` (reopen the app entry page without resetting application data)
- `browser reset-reference` (restore the clean seeded reference state after exploratory mutations)
- `browser click <page> <elementId>`
- `browser type <page> <elementId> <text>`
- `browser hover <page> <elementId>`
- `browser focus <page> <elementId>`
- `browser press <page> <key>`
- `browser scroll <page> <up|down>`
- `browser wait <page> [ms]`
- `browser back <page>` / `browser forward <page>`
- `browser upload <page> <elementId> <localFile>`
- `browser drag <page> <elementId> <targetElementId>`

Product specification

Purpose: <app_purpose>.

Core capabilities:

- <core_capability_1>
- <core_capability_2>
- ...

Secondary capabilities:

- <secondary_capability_1>
- <secondary_capability_2>

Treat this product specification as required but non-exhaustive coverage. Faithfully implement every item above, but do not limit the reconstruction to those items. Discover and reproduce all additional functionality and user-observable behavior available through the reference application.

```
## Implementation contract

- Work only inside `<repo_path>`.
- `app.js` must export `async function handle(req, res, context)`.
- Do not call `listen()`; the task-owned runtime owns all ports.
- Keep application state in `context.state` so the verifier can reset it.
- Use `context.readBody()`, `context.json()`, and `context.html()` as needed.
- The runtime hot-reloads repository modules on every request.
- Faithfully implement the routes, labels, forms, selectors, state transitions,
  downloads, and validation behavior observed in the reference.
```

G Model Pricing and Execution Time

G.1 Model Pricing

Table 8 lists list price per model, in \$ per 1M tokens, read from the LiteLLM `model_cost` table under the same bare model identifiers the agent harness uses (e.g. `claude-opus-5`, `gpt-5.6-sol`).

Table 8: **List price per model.** Input and output price in \$ per 1M tokens, from LiteLLM’s `model_cost` table.

Model	Input (\$/1M)	Output (\$/1M)
GPT-6 Astra	10.00	50.00
Claude Opus 5	5.00	25.00
GPT-5.6 Sol	4.00	20.00
GPT-5.3 Codex	1.75	14.00
Gemini 3.1 Pro Preview	2.00	12.00
Claude Sonnet 5	2.00	10.00
Grok 4.6	2.00	6.00
Gemini 3.6 Flash	0.75	3.75
Gemini 3.7 Flash	0.75	3.75

G.2 Execution Time

To give a concrete sense of the time required to run the benchmark, we report one representative evaluation run. GPT-6 Astra at high reasoning effort completed all 300 tasks in approximately 557 minutes of wall-clock time (9 hours 17 minutes) using four CPU servers. Each server had two Intel Xeon Gold 6238R processors at 2.20 GHz (56 physical cores, 112 logical CPUs) and ran up to eight trials concurrently, for a maximum of 32 concurrent trials. Each trial was allocated six logical CPUs, split evenly between the current and reference applications, with verification reusing the current application’s allocation. The servers hosted the applications, browsers, and verification processes, while model inference was served through a remote API.

Wall-clock time is measured from the earliest trial start to the final trial completion and includes per-trial setup, agent execution, verification, and API waiting time, but excludes one-time environment provisioning. Individual trials took 48.2 minutes on average, with a median of 43.1 minutes. This example is intended to provide a practical estimate of benchmark runtime rather than a measure of standalone model-inference speed.

H More Experimental Results

H.1 Results by Mask Scope

Every task in ProgramDistill-300 carries one of two mask scopes. A logic-only task removes the behavior behind an interface that remains unchanged, so the agent restores handlers, endpoints, queries, and state updates behind existing controls. A logic-and-UI task also removes the UI owned by the feature, requiring the agent to reconstruct both the observed interface and its underlying behavior. Table 9 compares performance across these two settings using binary and chain scores.

Astra achieves 92.9% binary success on logic-only tasks and 76.9% on logic-and-UI tasks, a 16.0-point gap. The chain score gives partial credit for behaviors recovered before the first failure in a cumulative task. Under this metric, Astra achieves 96.2% on logic-only tasks and 84.9% on logic-and-UI tasks, a 11.3-point gap. The scope comparison therefore captures both complete task success and the extent of partial restoration.

Table 9: **Performance by mask scope on ProgramDistill-300.** Logic-only tasks mask the behavior behind an unchanged interface, whereas logic-and-UI tasks also remove the UI owned by the feature. The suite contains 140 logic-only and 160 logic-and-UI tasks. Δ denotes the difference between logic-only and logic-and-UI performance.

Model	Binary score				Chain score			
	Overall	Logic-only	Logic-and-UI	Δ	Overall	Logic-only	Logic-and-UI	Δ
GPT-6 Astra	84.3	92.9	76.9	16.0	90.2	96.2	84.9	11.3
Claude Opus 5	68.7	80.0	58.8	21.2	75.2	85.7	66.0	19.7
GPT-5.6 Sol	60.7	76.4	46.9	29.6	68.2	81.5	56.5	25.0
Grok 4.6	48.3	64.3	34.4	29.9	57.1	72.7	43.5	29.2
Claude Sonnet 5	47.3	65.0	31.9	33.1	57.5	73.9	43.1	30.8
GPT-5.3 Codex	45.7	59.6	33.4	26.2	53.7	67.4	41.7	25.7
Gemini 3.7 Flash	45.3	61.4	31.2	30.2	53.2	69.5	38.9	30.6
Gemini 3.6 Flash	29.7	37.9	22.5	15.4	39.6	49.6	30.9	18.7
Gemini 3.1 Pro Preview	22.3	31.1	14.7	16.4	30.5	41.6	20.8	20.8

H.2 Results Under the Chain Score

The binary score gives credit only when all targets in a cumulative task pass, while the chain score credits the fraction recovered before the first failure. We compare these scores for the two reconstruction settings.

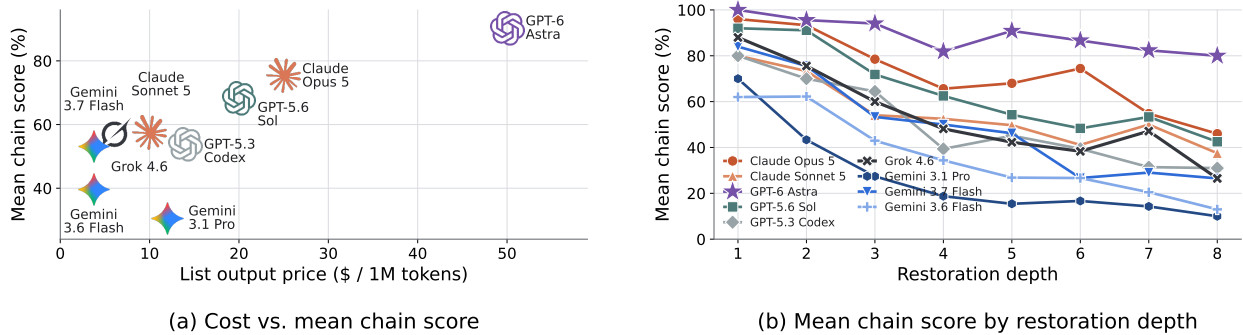


Figure 18: **Partial-application reconstruction performance under the chain score.** The chain-score counterpart of Figure 7. Results cover all 300 tasks in ProgramDistill-300, including both atomic and cumulative tasks. (a) Mean chain score versus list output price. (b) Mean chain score by restoration depth r_L . The decline with depth is shallower than under the binary score because partial restoration still earns credit, but the overall depth effect remains.

Partial-Application Reconstruction. For the setting in Section 3.2, Figure 18 is the chain-score counterpart of Figure 7, covering all 300 tasks in ProgramDistill-300. Table 10 compares binary and chain scores on the 250 cumulative repair tasks. Astra has the highest scores under both metrics: 81.2% binary and 88.2% chain score on

Table 10: **Partial-application reconstruction under binary and chain scores.** Mean scores (%) over the 250 cumulative tasks of ProgramDistill-300.

Model	Binary score	Chain score
GPT-6 Astra	81.2	88.2
Claude Opus 5	63.2	71.1
GPT-5.6 Sol	54.4	63.4
Claude Sonnet 5	40.8	53.0
Grok 4.6	40.4	50.9
GPT-5.3 Codex	38.8	48.4
Gemini 3.7 Flash	37.6	47.0
Gemini 3.6 Flash	23.2	35.2
Gemini 3.1 Pro Preview	12.8	22.6

cumulative tasks. The gap reflects cases where part of the restoration lineage is recovered without a complete task pass. Opus 5 increases from 63.2% to 71.1%, and Sol from 54.4% to 63.4%.

Full-Application Reconstruction. For the setting in Section 3.3, Table 11 reports chain scores in the same model and application order as Table 2. Mean binary/chain scores are 49.15/64.58% for Astra, 28.81/47.42% for Opus 5, and 21.07/34.22% for Sol. The larger gaps between the scores indicate that agents recover workflow prefixes more often than complete workflows.

 Table 11: **Full-application reconstruction under the chain score (%).** The chain-score counterpart of Table 2. The Mean column averages all evaluated cumulative workflows, and bold indicates the best result among the three models.

Model	Reactive Resume	AWS Console	Baserow	BudgetWise	CareerLink	MailHub	Jira (Oldboyxx)	Trello (Knowankit)	StreamView	TeamChat	Trello (Vdevired)	Weights & Biases	Mean
GPT-6 Astra (max)	68.5	45.8	36.7	56.0	65.3	89.0	85.6	73.2	91.4	47.0	44.3	80.9	64.58
Opus 5 (max)	43.7	35.4	35.3	39.8	41.4	93.9	93.3	73.9	28.0	42.9	29.6	35.3	47.42
GPT-5.6 Sol (max)	16.8	8.3	29.0	21.5	12.4	82.0	87.5	76.7	32.2	35.6	42.6	9.1	34.22

H.3 Does Context Length Explain the Depth Effect?

Deeper tasks generally expose agents to longer contexts, raising an alternative explanation for the performance decline in Section 3.2. Models may perform worse simply because they operate over longer contexts rather than because they must compose more dependent repairs. We examine this possibility in three ways using mean prompt size per step as a measure of context exposure across trajectories from all nine models.

First, the growth in context length does not closely track the decline in performance. The median prompt size per step increases from 47.5k tokens at restoration depth 1 to 107.5k at depth 8, a factor of 2.3, while the mean binary score falls from 83.4% to 23.1%. Most of the increase in context length occurs by depth 4. Beyond that point, context length changes little while performance continues to decline.

 Table 12: **Mean binary score by restoration depth among runs with similar context lengths.** Runs are grouped by their mean prompt size per step, and columns indicate restoration depth. This diagnostic uses the task outcomes paired with the analyzed trajectories.

Mean prompt size / step	Restoration depth							
	1	2	3	4	5	6	7	8
40–80k	82.6	77.1	65.0	51.8	48.1	46.4	45.3	24.2
80–120k	77.0	58.9	43.4	39.5	32.5	30.1	30.0	22.5
120–200k	54.5	68.8	33.9	33.3	30.0	25.4	23.2	23.1

Second, performance still declines with restoration depth among runs with similar context lengths. Within each context band in Table 12, binary score shows an overall downward trend as depth increases. From depth 1 to depth 8, mean binary score drops by 58.4 points in the 40–80k band, 54.5 points in the 80–120k band, and 31.5 points in the 120–200k band. Thus, deeper tasks remain substantially harder even at comparable context lengths.

Third, within each restoration depth, prompt size has only a modest negative Pearson correlation with binary score, ranging from -0.20 to -0.05 . After centering both prompt size and binary score within each task to account for task differences, the pooled correlation is weakly positive ($+0.11$).

Together, these analyses show that the depth-related performance decline cannot be explained by longer contexts alone and are consistent with the increasing difficulty of composing multiple dependent repairs.

H.4 Agent Interaction Statistics

Partial-Application Reconstruction. Table 13 provides the full interaction statistics supporting Figure 9 in Section 3.2.2: the minimum, mean, and maximum counts of agent steps, observation steps, and edit/write steps for each model. The lower panel groups the same trajectories by restoration depth and reports mean counts across all nine models.

Table 13: **Agent interaction behavior.** The upper panel reports min, mean, and max interaction statistics for each model on ProgramDistill-300; the lower panel reports per-trajectory means by restoration depth, pooling all nine models. An observation step contains a request for application state through an observe command, a state-returning browser interaction, or a supported direct browser or API check of the current app. Each agent step counts at most once, even when it targets both applications.

Model	Steps			Observation steps			Edit/write steps		
	Min	Mean	Max	Min	Mean	Max	Min	Mean	Max
GPT-6 Astra	42	229.2	757	35	191.3	581	1	9.9	58
Claude Opus 5	29	209.3	1000	1	95.3	329	0	23.5	117
Claude Sonnet 5	27	222.4	522	8	107.2	364	1	15.7	53
GPT-5.6 Sol	42	199.4	377	15	101.2	236	1	25.8	85
GPT-5.3 Codex	58	196.4	342	11	86.7	195	0	17.1	60
Grok 4.6	41	160.9	370	2	65.9	173	1	18.5	69
Gemini 3.1 Pro	35	159.6	858	2	46.3	152	1	24.2	106
Gemini 3.7 Flash	50	206.8	626	2	82.5	218	1	13.5	59
Gemini 3.6 Flash	55	180.2	386	9	68.9	184	0	16.8	62

Restoration depth	Mean steps		Mean observation steps		Mean edit/write steps	
1	124.0		61.2		7.5	
2	164.0		75.9		12.4	
3	187.0		92.8		17.0	
4	216.0		99.9		21.2	
5	225.9		110.5		23.0	
6	230.8		110.5		24.3	
7	235.7		110.8		24.5	
8	250.6		121.0		27.2	

Full-Application Reconstruction. Table 14 reports agent steps by application for the full-application runs in Table 2. Astra averages 719.4 steps, compared with 627.5 for Opus 5 and 719.0 for Sol.

Table 14: **Agent steps per full-application reconstruction run.**

Model	Reactive Resume	AWS Console	Baserow	BudgetWise	CareerLink	MailHub	Jira (Oldboyxx)	Trello (Knowankit)	StreamView	TeamChat	Trello (Vdevired)	Weights & Biases	Mean
GPT-6 Astra (max)	853	1921	923	292	824	421	489	434	895	385	700	496	719.4
Opus 5 (max)	620	837	572	663	940	417	371	595	601	608	533	773	627.5
GPT-5.6 Sol (max)	1165	856	860	580	861	348	598	446	990	488	470	966	719.0